IE-494 Lab 09 — Study of Compaction in RocksDB

Course: IE-494 Big Data Processing · Autumn 2026
Instructor: PM Jat (`pm_jat@dau.ac.in`)
Student: Aditya Vaish · ID `202401401`
Submitted: 26 October 2026
Stack: RocksJava `rocksdbjni 10.10.1.1`, Java (release 17), Maven, macOS arm64 (8 cores, 8 GB)

1. Assignment

1. Dump Lab 08's `CORDERS` (each order with its line items embedded) to `corders.json`.
2. Iterate through every order in that file and add it to a new RocksDB database `CORDERS`. Catch every flush and compaction, and for each one print the SSTables before and after it: file name, level, size, smallest key, largest key.
3. After all orders are added, print the final SSTables the same way and also dump the index of each SSTable.
4. Repeat steps 2 and 3 for **leveled** and **universal** compaction.

2. Data and settings

Input. TPC-H scale factor 1, built with Lab 08's code: 1,500,000 orders and 6,001,215 line items merged into `lab8/db/CORDERS_zstd`. `corders.json` is one JSON array with one order per line, 2,583,603,406 bytes, key range `0000000001 .. 0006000000`. Keys are Lab 08's 10-digit zero-padded `O_ORDERKEY`, so byte order matches numeric order.

option	value	why
write_buffer_size × max_write_buffer_number	128 MiB × 2	about 20 flushes for 2.6 GB, so the event log stays readable
level0_file_num_compaction_trigger	4	default
target_file_size_base / max_bytes_for_level_base	64 MiB / 256 MiB	default 64 MiB target; 256 MiB level base (same as Lab 08)
level_compaction_dynamic_level_bytes (leveled)	true	default since RocksDB 8.4
universal options	size_ratio=1, min_merge_width=2, max_size_amplification_percent=200	defaults, set explicitly
compression	LZ4 (every level)	fast; one codec for all levels
block_size	64 KiB	about 37 orders per data block, so each index has about 2,100 entries per file instead of about 40,000
format_version / index compression	5 / off	fixed on-disk layout for the index parser
max_background_jobs	2	one flush thread and one compaction thread, so the before/after snapshots are easy to follow

option	value	why
writes	WriteBatch of 2,000, WAL off	same as Lab 08

3. How it works

- **Dump** (`CordersJson.dump`): iterate Lab 08's CORDERS with a read-only `RocksIterator` and stream each value to `corders.json`.
- **Load** (`CompactionStudy.run`): read `corders.json` line by line, take the key from the `O_ORDERKEY` field, and `put` the order document into a fresh `CORDERS` in batches of 2,000. At the end, one manual flush; then poll `rocksdb.num-running-compactions`, `compaction-pending` and the flush counters until RocksDB has been idle for 1 s (RocksJava has no `WaitForCompact`).
- **Catching events** (`SstEventListener`): an `AbstractEventListener` that turns on only the four callbacks `onFlushBegin`, `onFlushCompleted`, `onCompactionBegin` and `onCompactionCompleted`. Each callback calls `db.getLiveFilesMetaData()` and prints every live SSTable (name, level, size, smallest key, largest key). *Begin* gives the "before" list and *Completed* gives the "after" list. Compaction inputs are marked ∗ and new files +. `CompactionJobInfo` also gives the reason, the input and output levels, and the bytes read and written. A job whose output files are the same as its inputs is a *trivial move*: the files are moved to a new level in the MANIFEST without being rewritten.
- **Index dump** (`SstIndex`): RocksJava cannot read an index block, so the program decodes the block-based table format itself. It reads the 53-byte footer (magic `0x88e241b785f4cff7`) and the index handle, then decodes each index entry (shared and non-shared key length, the key bytes, and a BlockHandle varint pair or a size delta). It checks the result against the `TableProperties` from `SstFileReader`: exactly one entry per data block, blocks contiguous from offset 0, and separator keys in ascending order. Every file passed.
- Run with `bash lab9/run_all.sh`. It uses `mvn exec:exec` to run a separate JVM, because under `exec:java` RocksDB's native callback threads cannot find the RocksJava classes (`NoClassDefFoundError: org/rocksdb/FlushJobInfo`).

4. Results

	leveled	universal
orders loaded	1,500,000	1,500,000
flushes	20	20
compactions	7	3
trivial moves / merges	7 / 0	0 / 3
bytes flushed to L0	763,020,785	763,020,785
bytes rewritten by compaction	0	1,223,950,521
write amplification	1.00	2.60
final SSTables	20	13
final layout	L0=2 L5=5 L6=13	L0=1 L6=12
final SST bytes	763,020,785	760,621,019
load time (writes)	4.75 s	6.46 s
last flush + compactions settle	1.26 s	5.97 s

4.1 Leveled compactions

#	reason	levels	kind	in	out	bytes read	bytes written	secs
1	kLevelL0FilesNum	L0 -> L6	trivial move	1	1	0	0	0.006
2	kLevelL0FilesNum	L0 -> L6	trivial move	4	4	0	0	0.004
3	kLevelL0FilesNum	L0 -> L6	trivial move	4	4	0	0	0.003
4	kLevelL0FilesNum	L0 -> L5	trivial move	1	1	0	0	0.007
5	kLevelL0FilesNum	L0 -> L5	trivial move	4	4	0	0	0.004
6	kLevelL0FilesNum	L0 -> L5	trivial move	4	4	0	0	0.005
7	kLevelMaxLevelSize	L5 -> L6	trivial move	4	4	0	0	0.004

4.2 Universal compactions

#	reason	levels	kind	in	out	bytes read	bytes written	secs
1	kUniversalSizeAmplification	L0 -> L6	merge	4	3	158,659,889	158,153,336	1.578
2	kUniversalSizeRatio	L0 -> L6	merge	7	5	315,862,874	315,364,104	4.388
3	kUniversalSizeRatio	L0 -> L6	merge	16	12	751,827,524	750,433,081	4.641

5. Observations

- **Flushes are the same for both styles.** Each style made 20 flushes (19 because the write buffer was full, plus 1 final manual flush). Each full memtable became one L0 SSTable of about 39.6 MB holding 78,000 orders (about 134 MB of JSON, LZ4-compressed to about 30%).
- **Leveled: every compaction was a trivial move** (7 compactions, 0 bytes rewritten, write amplification 1.00). `corders.json` is in key order, so each new L0 file covers keys above everything already stored. It overlaps nothing in the target level, so RocksDB moves the file to a new level without rewriting it (same file name before and after). With dynamic level bytes, L0 first moves straight to L6. Once L6 is larger than 256 MiB the base level becomes L5, and when L5 goes over its target a `kLevelMaxLevelSize` compaction moves L5 files down to L6. The final 20 files are still exactly the 20 flush outputs.
- **Universal: real merges, even for sorted input** (3 merges, 1,223,950,521 bytes rewritten, write amplification 2.60). The first compaction is `kUniversalSizeAmplification`: 4 sorted runs make the size amplification 300%, above the 200% limit. The later ones are `kUniversalSizeRatio`. Each compaction rewrites the previous merged run together with the L0 files flushed since, so the inputs keep growing (4 → 7 → 16 files). Output to L6 is cut into 64 MiB files (target_file_size_base). Universal compaction only does trivial moves when `allow_trivial_move` is turned on (it is off by default), so even sorted runs that do not overlap were rewritten.
- **Cost of the merges.** With one compaction thread, the long universal merges let L0 grow to 15 files (leveled peaked at 4), and the final settle took 6.0 s against 1.3 s for leveled. Universal ends with fewer, larger files (L0=1 L6=12: two sorted runs, below the trigger of 4). Leveled ends with L0=2 L5=5 L6=13.
- **Index.** Each 64 KiB data block (about 19 KB after LZ4) has one index entry: *separator key → (offset, size)*. A separator is at least every key in its block and smaller than the first key of the next block, and RocksDB shortens it where it can (for example `000560829` or `00056089` instead of a full 10-digit key). The last entry in each file is the file's largest key. The index stores user keys (no sequence numbers) and delta-encoded block handles, so a 78,000-order file has a 47 KB index of about 2,100 entries.

6. Program outputs

6.1 Console log (dump, load progress, comparison)

`output/run_log.txt`.

```
======== Dump CORDERS -> corders.json ========
source rocksdb : /Users/avaish/IE-494/lab8/db/CORDERS_zstd
json file      : /Users/avaish/IE-494/lab9/data/corders.json
orders written : 1,500,000
document bytes : 2,580,603,403
file size      : 2,583,603,406 bytes (2463.9 MiB)
key range      : 0000000001 .. 0006000000
dump time      : 5.680 s

======== LEVELED compaction: load corders.json -> /Users/avaish/IE-494/lab9/db/leveled/CORDERS ========
  orders 250,000 in 0.7s, live SSTables L0=2
  orders 500,000 in 1.4s, live SSTables L6=5
  orders 750,000 in 2.1s, live SSTables L0=3 L6=5
  orders 1,000,000 in 3.1s, live SSTables L0=2 L6=9
  orders 1,250,000 in 4.0s, live SSTables L0=1 L5=5 L6=9
  orders 1,500,000 in 4.8s, live SSTables L5=5 L6=13
flushes               : 20 (763,020,785 bytes written to L0)
compactions           : 7 (7 trivial moves, 0 merges)
compaction read bytes : 0
compaction write bytes: 0
write amplification   : 1.00  ((flush + compaction writes) / flush writes)

======== LEVELED final SSTables (/Users/avaish/IE-494/lab9/db/leveled/CORDERS) ========
(final SSTables: see the final_sstables.txt sections below)
======== UNIVERSAL compaction: load corders.json -> /Users/avaish/IE-494/lab9/db/universal/CORDERS ========
  orders 250,000 in 0.5s, live SSTables L0=2
  orders 500,000 in 1.1s, live SSTables L0=5
  orders 750,000 in 2.3s, live SSTables L0=8
  orders 1,000,000 in 3.6s, live SSTables L0=7 L6=3
  orders 1,250,000 in 5.3s, live SSTables L0=11 L6=3
  orders 1,500,000 in 6.5s, live SSTables L0=14 L6=3
flushes               : 20 (763,020,785 bytes written to L0)
compactions           : 3 (0 trivial moves, 3 merges)
compaction read bytes : 1,226,350,287
compaction write bytes: 1,223,950,521
write amplification   : 2.60  ((flush + compaction writes) / flush writes)

======== UNIVERSAL final SSTables (/Users/avaish/IE-494/lab9/db/universal/CORDERS) ========
(final SSTables: see the final_sstables.txt sections below)
======== Leveled vs Universal ========
style       flushes compactions trivial  merges  write-amp  final-sst  levels             final-bytes
leveled          20           7       7       0       1.00         20  L0=2 L5=5 L6=13     763,020,785
universal        20           3       0       3       2.60         13  L0=1 L6=12         760,621,019
summary: /Users/avaish/IE-494/lab9/output/summary.tsv
```

6.2 Leveled: every flush and compaction (`output/leveled/events.txt`)

```
LEVELED compaction: every flush and compaction while loading corders.json
rocksdb path: /Users/avaish/IE-494/lab9/db/leveled/CORDERS
options: compaction_style=kCompactionStyleLevel (level_compaction_dynamic_level_bytes=true), write_buffer_size=128 MiB
x 2, level0_file_num_compaction_trigger=4, target_file_size_base=64 MiB, max_bytes_for_level_base=256 MiB, compression=
LZ4, block_size=64 KiB, format_version=5, index compression off, max_background_jobs=2, WAL off, batch=2,000
'*' marks compaction inputs, '+' marks files written (or moved) by the job.

[event 001  t=  0.329s] FLUSH BEGIN       job=2 reason=WRITE_BUFFER_FULL
  SSTables before: 0 files, 0 bytes, no levels

[event 002  t=  0.551s] FLUSH COMPLETED   job=2 flush #1 wrote 000008.sst (39,646,772 bytes) to L0, seqno 1..78,000
  SSTables after: 1 files, 39,646,772 bytes, L0=1
      file-name   level     file-size  smallest-key largest-key
    + 000008.sst  L0       39,646,772  0000000001   0000312000

[event 003  t=  0.552s] FLUSH BEGIN       job=3 reason=WRITE_BUFFER_FULL
  SSTables before: 1 files, 39,646,772 bytes, L0=1
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000

[event 004  t=  0.766s] FLUSH COMPLETED   job=3 flush #2 wrote 000009.sst (39,719,410 bytes) to L0, seqno 78,001..156,0
00
  SSTables after: 2 files, 79,366,182 bytes, L0=2
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
```

```
    + 000009.sst  L0       39,719,410  0000312001   0000624000

[event 005  t=  0.767s] FLUSH BEGIN       job=4 reason=WRITE_BUFFER_FULL
  SSTables before: 2 files, 79,366,182 bytes, L0=2
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000

[event 006  t=  0.963s] FLUSH COMPLETED   job=4 flush #3 wrote 000010.sst (39,670,846 bytes) to L0, seqno 156,001..234,
000
  SSTables after: 3 files, 119,037,028 bytes, L0=3
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
    + 000010.sst  L0       39,670,846  0000624001   0000936000

[event 007  t=  0.964s] FLUSH BEGIN       job=5 reason=WRITE_BUFFER_FULL
  SSTables before: 3 files, 119,037,028 bytes, L0=3
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000

[event 008  t=  1.196s] FLUSH COMPLETED   job=5 flush #4 wrote 000011.sst (39,622,861 bytes) to L0, seqno 234,001..312,
000
  SSTables after: 4 files, 158,659,889 bytes, L0=4
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
    + 000011.sst  L0       39,622,861  0000936001   0001248000

[event 009  t=  1.199s] COMPACTION BEGIN  job=6 reason=kLevelL0FilesNum L0 -> L6, 1 input files
  SSTables before ('*' = input): 4 files, 158,659,889 bytes, L0=4
      file-name   level     file-size  smallest-key largest-key
    * 000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000

[event 010  t=  1.199s] FLUSH BEGIN       job=7 reason=WRITE_BUFFER_FULL
  SSTables before: 4 files, 158,659,889 bytes, L0=4
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000

[event 011  t=  1.205s] COMPACTION DONE   job=6 compaction #1 L0 -> L6 TRIVIAL MOVE (no rewrite): 1 in, 1 out, read 0,
wrote 0 bytes
  SSTables after ('+' = output): 4 files, 158,659,889 bytes, L0=3 L6=1
      file-name   level     file-size  smallest-key largest-key
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000
    + 000008.sst  L6       39,646,772  0000000001   0000312000

[event 012  t=  1.391s] COMPACTION BEGIN  job=8 reason=kLevelL0FilesNum L0 -> L6, 4 input files
  SSTables before ('*' = input): 5 files, 198,297,845 bytes, L0=4 L6=1
      file-name   level     file-size  smallest-key largest-key
    * 000009.sst  L0       39,719,410  0000312001   0000624000
    * 000010.sst  L0       39,670,846  0000624001   0000936000
    * 000011.sst  L0       39,622,861  0000936001   0001248000
    * 000012.sst  L0       39,637,956  0001248001   0001560000
      000008.sst  L6       39,646,772  0000000001   0000312000

[event 013  t=  1.392s] FLUSH COMPLETED   job=7 flush #5 wrote 000012.sst (39,637,956 bytes) to L0, seqno 312,001..390,
000
  SSTables after: 5 files, 198,297,845 bytes, L0=4 L6=1
      file-name   level     file-size  smallest-key largest-key
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000
    + 000012.sst  L0       39,637,956  0001248001   0001560000
      000008.sst  L6       39,646,772  0000000001   0000312000

[event 014  t=  1.393s] FLUSH BEGIN       job=9 reason=WRITE_BUFFER_FULL
  SSTables before: 5 files, 198,297,845 bytes, L0=4 L6=1
      file-name   level     file-size  smallest-key largest-key
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000
      000012.sst  L0       39,637,956  0001248001   0001560000
      000008.sst  L6       39,646,772  0000000001   0000312000
```

```
[event 015  t=  1.395s] COMPACTION DONE   job=8 compaction #2 L0 -> L6 TRIVIAL MOVE (no rewrite): 4 in, 4 out, read 0,
wrote 0 bytes
  SSTables after ('+' = output): 5 files, 198,297,845 bytes, L6=5
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L6       39,646,772  0000000001   0000312000
    + 000009.sst  L6       39,719,410  0000312001   0000624000
    + 000010.sst  L6       39,670,846  0000624001   0000936000
    + 000011.sst  L6       39,622,861  0000936001   0001248000
    + 000012.sst  L6       39,637,956  0001248001   0001560000

[event 016  t=  1.612s] FLUSH COMPLETED   job=9 flush #6 wrote 000013.sst (38,771,506 bytes) to L0, seqno 390,001..466,
000
  SSTables after: 6 files, 237,069,351 bytes, L0=1 L6=5
      file-name   level     file-size  smallest-key largest-key
    + 000013.sst  L0       38,771,506  0001560001   0001864000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000

[event 017  t=  1.613s] FLUSH BEGIN       job=10 reason=WRITE_BUFFER_FULL
  SSTables before: 6 files, 237,069,351 bytes, L0=1 L6=5
      file-name   level     file-size  smallest-key largest-key
      000013.sst  L0       38,771,506  0001560001   0001864000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000

[event 018  t=  1.809s] FLUSH COMPLETED   job=10 flush #7 wrote 000014.sst (39,630,369 bytes) to L0, seqno 466,001..54
4,000
  SSTables after: 7 files, 276,699,720 bytes, L0=2 L6=5
      file-name   level     file-size  smallest-key largest-key
      000013.sst  L0       38,771,506  0001560001   0001864000
    + 000014.sst  L0       39,630,369  0001864001   0002176000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000

[event 019  t=  1.810s] FLUSH BEGIN       job=11 reason=WRITE_BUFFER_FULL
  SSTables before: 7 files, 276,699,720 bytes, L0=2 L6=5
      file-name   level     file-size  smallest-key largest-key
      000013.sst  L0       38,771,506  0001560001   0001864000
      000014.sst  L0       39,630,369  0001864001   0002176000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000

[event 020  t=  2.023s] FLUSH COMPLETED   job=11 flush #8 wrote 000015.sst (39,669,707 bytes) to L0, seqno 544,001..62
2,000
  SSTables after: 8 files, 316,369,427 bytes, L0=3 L6=5
      file-name   level     file-size  smallest-key largest-key
      000013.sst  L0       38,771,506  0001560001   0001864000
      000014.sst  L0       39,630,369  0001864001   0002176000
    + 000015.sst  L0       39,669,707  0002176001   0002488000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000

[event 021  t=  2.024s] FLUSH BEGIN       job=12 reason=WRITE_BUFFER_FULL
  SSTables before: 8 files, 316,369,427 bytes, L0=3 L6=5
      file-name   level     file-size  smallest-key largest-key
      000013.sst  L0       38,771,506  0001560001   0001864000
      000014.sst  L0       39,630,369  0001864001   0002176000
      000015.sst  L0       39,669,707  0002176001   0002488000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000

[event 022  t=  2.204s] COMPACTION BEGIN  job=13 reason=kLevelL0FilesNum L0 -> L6, 4 input files
  SSTables before ('*' = input): 9 files, 356,015,082 bytes, L0=4 L6=5
      file-name   level     file-size  smallest-key largest-key
    * 000013.sst  L0       38,771,506  0001560001   0001864000
    * 000014.sst  L0       39,630,369  0001864001   0002176000
    * 000015.sst  L0       39,669,707  0002176001   0002488000
```

```
    * 000016.sst  L0       39,645,655  0002488001   0002800000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000

[event 023  t=  2.205s] FLUSH COMPLETED   job=12 flush #9 wrote 000016.sst (39,645,655 bytes) to L0, seqno 622,001..70
0,000
  SSTables after: 9 files, 356,015,082 bytes, L0=4 L6=5
      file-name   level     file-size  smallest-key largest-key
      000013.sst  L0       38,771,506  0001560001   0001864000
      000014.sst  L0       39,630,369  0001864001   0002176000
      000015.sst  L0       39,669,707  0002176001   0002488000
    + 000016.sst  L0       39,645,655  0002488001   0002800000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000

[event 024  t=  2.207s] COMPACTION DONE   job=13 compaction #3 L0 -> L6 TRIVIAL MOVE (no rewrite): 4 in, 4 out, read 0,
wrote 0 bytes
  SSTables after ('+' = output): 9 files, 356,015,082 bytes, L6=9
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
    + 000013.sst  L6       38,771,506  0001560001   0001864000
    + 000014.sst  L6       39,630,369  0001864001   0002176000
    + 000015.sst  L6       39,669,707  0002176001   0002488000
    + 000016.sst  L6       39,645,655  0002488001   0002800000

[event 025  t=  2.207s] FLUSH BEGIN       job=14 reason=WRITE_BUFFER_FULL
  SSTables before: 9 files, 356,015,082 bytes, L6=9
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 026  t=  2.427s] FLUSH COMPLETED   job=14 flush #10 wrote 000017.sst (39,646,656 bytes) to L0, seqno 700,001..77
8,000
  SSTables after: 10 files, 395,661,738 bytes, L0=1 L6=9
      file-name   level     file-size  smallest-key largest-key
    + 000017.sst  L0       39,646,656  0002800001   0003112000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 027  t=  2.433s] FLUSH BEGIN       job=15 reason=WRITE_BUFFER_FULL
  SSTables before: 10 files, 395,661,738 bytes, L0=1 L6=9
      file-name   level     file-size  smallest-key largest-key
      000017.sst  L0       39,646,656  0002800001   0003112000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 028  t=  3.025s] FLUSH COMPLETED   job=15 flush #11 wrote 000018.sst (39,714,036 bytes) to L0, seqno 778,001..85
6,000
  SSTables after: 11 files, 435,375,774 bytes, L0=2 L6=9
      file-name   level     file-size  smallest-key largest-key
      000017.sst  L0       39,646,656  0002800001   0003112000
    + 000018.sst  L0       39,714,036  0003112001   0003424000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
```

```
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 029  t=  3.027s] FLUSH BEGIN       job=16 reason=WRITE_BUFFER_FULL
  SSTables before: 11 files, 435,375,774 bytes, L0=2 L6=9
      file-name   level     file-size  smallest-key largest-key
      000017.sst  L0       39,646,656  0002800001   0003112000
      000018.sst  L0       39,714,036  0003112001   0003424000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 030  t=  3.239s] FLUSH COMPLETED   job=16 flush #12 wrote 000019.sst (39,733,807 bytes) to L0, seqno 856,001..93
4,000
  SSTables after: 12 files, 475,109,581 bytes, L0=3 L6=9
      file-name   level     file-size  smallest-key largest-key
      000017.sst  L0       39,646,656  0002800001   0003112000
      000018.sst  L0       39,714,036  0003112001   0003424000
    + 000019.sst  L0       39,733,807  0003424001   0003736000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 031  t=  3.254s] FLUSH BEGIN       job=17 reason=WRITE_BUFFER_FULL
  SSTables before: 12 files, 475,109,581 bytes, L0=3 L6=9
      file-name   level     file-size  smallest-key largest-key
      000017.sst  L0       39,646,656  0002800001   0003112000
      000018.sst  L0       39,714,036  0003112001   0003424000
      000019.sst  L0       39,733,807  0003424001   0003736000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 032  t=  3.520s] FLUSH COMPLETED   job=17 flush #13 wrote 000020.sst (39,634,266 bytes) to L0, seqno 934,001..1,
012,000
  SSTables after: 13 files, 514,743,847 bytes, L0=4 L6=9
      file-name   level     file-size  smallest-key largest-key
      000017.sst  L0       39,646,656  0002800001   0003112000
      000018.sst  L0       39,714,036  0003112001   0003424000
      000019.sst  L0       39,733,807  0003424001   0003736000
    + 000020.sst  L0       39,634,266  0003736001   0004048000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 033  t=  3.520s] COMPACTION BEGIN  job=18 reason=kLevelL0FilesNum L0 -> L5, 1 input files
  SSTables before ('*' = input): 13 files, 514,743,847 bytes, L0=4 L6=9
      file-name   level     file-size  smallest-key largest-key
    * 000017.sst  L0       39,646,656  0002800001   0003112000
      000018.sst  L0       39,714,036  0003112001   0003424000
      000019.sst  L0       39,733,807  0003424001   0003736000
      000020.sst  L0       39,634,266  0003736001   0004048000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
```

```
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 034  t=  3.521s] FLUSH BEGIN       job=19 reason=WRITE_BUFFER_FULL
  SSTables before: 13 files, 514,743,847 bytes, L0=4 L6=9
      file-name   level     file-size  smallest-key largest-key
      000017.sst  L0       39,646,656  0002800001   0003112000
      000018.sst  L0       39,714,036  0003112001   0003424000
      000019.sst  L0       39,733,807  0003424001   0003736000
      000020.sst  L0       39,634,266  0003736001   0004048000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 035  t=  3.527s] COMPACTION DONE   job=18 compaction #4 L0 -> L5 TRIVIAL MOVE (no rewrite): 1 in, 1 out, read 0,
wrote 0 bytes
  SSTables after ('+' = output): 13 files, 514,743,847 bytes, L0=3 L5=1 L6=9
      file-name   level     file-size  smallest-key largest-key
      000018.sst  L0       39,714,036  0003112001   0003424000
      000019.sst  L0       39,733,807  0003424001   0003736000
      000020.sst  L0       39,634,266  0003736001   0004048000
    + 000017.sst  L5       39,646,656  0002800001   0003112000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 036  t=  3.754s] COMPACTION BEGIN  job=20 reason=kLevelL0FilesNum L0 -> L5, 4 input files
  SSTables before ('*' = input): 14 files, 554,460,365 bytes, L0=4 L5=1 L6=9
      file-name   level     file-size  smallest-key largest-key
    * 000018.sst  L0       39,714,036  0003112001   0003424000
    * 000019.sst  L0       39,733,807  0003424001   0003736000
    * 000020.sst  L0       39,634,266  0003736001   0004048000
    * 000021.sst  L0       39,716,518  0004048001   0004360000
      000017.sst  L5       39,646,656  0002800001   0003112000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 037  t=  3.755s] FLUSH COMPLETED   job=19 flush #14 wrote 000021.sst (39,716,518 bytes) to L0, seqno 1,012,001..
1,090,000
  SSTables after: 14 files, 554,460,365 bytes, L0=4 L5=1 L6=9
      file-name   level     file-size  smallest-key largest-key
      000018.sst  L0       39,714,036  0003112001   0003424000
      000019.sst  L0       39,733,807  0003424001   0003736000
      000020.sst  L0       39,634,266  0003736001   0004048000
    + 000021.sst  L0       39,716,518  0004048001   0004360000
      000017.sst  L5       39,646,656  0002800001   0003112000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 038  t=  3.758s] COMPACTION DONE   job=20 compaction #5 L0 -> L5 TRIVIAL MOVE (no rewrite): 4 in, 4 out, read 0,
wrote 0 bytes
  SSTables after ('+' = output): 14 files, 554,460,365 bytes, L5=5 L6=9
      file-name   level     file-size  smallest-key largest-key
      000017.sst  L5       39,646,656  0002800001   0003112000
    + 000018.sst  L5       39,714,036  0003112001   0003424000
    + 000019.sst  L5       39,733,807  0003424001   0003736000
```

```
    + 000020.sst  L5       39,634,266  0003736001   0004048000
    + 000021.sst  L5       39,716,518  0004048001   0004360000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 039  t=  3.758s] FLUSH BEGIN       job=21 reason=WRITE_BUFFER_FULL
  SSTables before: 14 files, 554,460,365 bytes, L5=5 L6=9
      file-name   level     file-size  smallest-key largest-key
      000017.sst  L5       39,646,656  0002800001   0003112000
      000018.sst  L5       39,714,036  0003112001   0003424000
      000019.sst  L5       39,733,807  0003424001   0003736000
      000020.sst  L5       39,634,266  0003736001   0004048000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 040  t=  4.073s] FLUSH COMPLETED   job=21 flush #15 wrote 000022.sst (39,651,458 bytes) to L0, seqno 1,090,001..
1,168,000
  SSTables after: 15 files, 594,111,823 bytes, L0=1 L5=5 L6=9
      file-name   level     file-size  smallest-key largest-key
    + 000022.sst  L0       39,651,458  0004360001   0004672000
      000017.sst  L5       39,646,656  0002800001   0003112000
      000018.sst  L5       39,714,036  0003112001   0003424000
      000019.sst  L5       39,733,807  0003424001   0003736000
      000020.sst  L5       39,634,266  0003736001   0004048000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 041  t=  4.074s] FLUSH BEGIN       job=22 reason=WRITE_BUFFER_FULL
  SSTables before: 15 files, 594,111,823 bytes, L0=1 L5=5 L6=9
      file-name   level     file-size  smallest-key largest-key
      000022.sst  L0       39,651,458  0004360001   0004672000
      000017.sst  L5       39,646,656  0002800001   0003112000
      000018.sst  L5       39,714,036  0003112001   0003424000
      000019.sst  L5       39,733,807  0003424001   0003736000
      000020.sst  L5       39,634,266  0003736001   0004048000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 042  t=  4.301s] FLUSH COMPLETED   job=22 flush #16 wrote 000023.sst (39,701,916 bytes) to L0, seqno 1,168,001..
1,246,000
  SSTables after: 16 files, 633,813,739 bytes, L0=2 L5=5 L6=9
      file-name   level     file-size  smallest-key largest-key
      000022.sst  L0       39,651,458  0004360001   0004672000
    + 000023.sst  L0       39,701,916  0004672001   0004984000
      000017.sst  L5       39,646,656  0002800001   0003112000
      000018.sst  L5       39,714,036  0003112001   0003424000
      000019.sst  L5       39,733,807  0003424001   0003736000
      000020.sst  L5       39,634,266  0003736001   0004048000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
```

```
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 043  t=  4.301s] FLUSH BEGIN       job=23 reason=WRITE_BUFFER_FULL
  SSTables before: 16 files, 633,813,739 bytes, L0=2 L5=5 L6=9
      file-name   level     file-size  smallest-key largest-key
      000022.sst  L0       39,651,458  0004360001   0004672000
      000023.sst  L0       39,701,916  0004672001   0004984000
      000017.sst  L5       39,646,656  0002800001   0003112000
      000018.sst  L5       39,714,036  0003112001   0003424000
      000019.sst  L5       39,733,807  0003424001   0003736000
      000020.sst  L5       39,634,266  0003736001   0004048000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 044  t=  4.565s] FLUSH COMPLETED   job=23 flush #17 wrote 000024.sst (39,611,404 bytes) to L0, seqno 1,246,001..
1,324,000
  SSTables after: 17 files, 673,425,143 bytes, L0=3 L5=5 L6=9
      file-name   level     file-size  smallest-key largest-key
      000022.sst  L0       39,651,458  0004360001   0004672000
      000023.sst  L0       39,701,916  0004672001   0004984000
    + 000024.sst  L0       39,611,404  0004984001   0005296000
      000017.sst  L5       39,646,656  0002800001   0003112000
      000018.sst  L5       39,714,036  0003112001   0003424000
      000019.sst  L5       39,733,807  0003424001   0003736000
      000020.sst  L5       39,634,266  0003736001   0004048000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 045  t=  4.565s] FLUSH BEGIN       job=24 reason=WRITE_BUFFER_FULL
  SSTables before: 17 files, 673,425,143 bytes, L0=3 L5=5 L6=9
      file-name   level     file-size  smallest-key largest-key
      000022.sst  L0       39,651,458  0004360001   0004672000
      000023.sst  L0       39,701,916  0004672001   0004984000
      000024.sst  L0       39,611,404  0004984001   0005296000
      000017.sst  L5       39,646,656  0002800001   0003112000
      000018.sst  L5       39,714,036  0003112001   0003424000
      000019.sst  L5       39,733,807  0003424001   0003736000
      000020.sst  L5       39,634,266  0003736001   0004048000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 046  t=  4.789s] COMPACTION BEGIN  job=25 reason=kLevelL0FilesNum L0 -> L5, 4 input files
  SSTables before ('*' = input): 18 files, 713,078,155 bytes, L0=4 L5=5 L6=9
      file-name   level     file-size  smallest-key largest-key
    * 000022.sst  L0       39,651,458  0004360001   0004672000
    * 000023.sst  L0       39,701,916  0004672001   0004984000
    * 000024.sst  L0       39,611,404  0004984001   0005296000
    * 000025.sst  L0       39,653,012  0005296001   0005608000
      000017.sst  L5       39,646,656  0002800001   0003112000
      000018.sst  L5       39,714,036  0003112001   0003424000
      000019.sst  L5       39,733,807  0003424001   0003736000
      000020.sst  L5       39,634,266  0003736001   0004048000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
```

```
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 047  t=  4.790s] FLUSH COMPLETED   job=24 flush #18 wrote 000025.sst (39,653,012 bytes) to L0, seqno 1,324,001..
1,402,000
  SSTables after: 18 files, 713,078,155 bytes, L0=4 L5=5 L6=9
      file-name   level     file-size  smallest-key largest-key
      000022.sst  L0       39,651,458  0004360001   0004672000
      000023.sst  L0       39,701,916  0004672001   0004984000
      000024.sst  L0       39,611,404  0004984001   0005296000
    + 000025.sst  L0       39,653,012  0005296001   0005608000
      000017.sst  L5       39,646,656  0002800001   0003112000
      000018.sst  L5       39,714,036  0003112001   0003424000
      000019.sst  L5       39,733,807  0003424001   0003736000
      000020.sst  L5       39,634,266  0003736001   0004048000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 048  t=  4.793s] FLUSH BEGIN       job=26 reason=WRITE_BUFFER_FULL
  SSTables before: 18 files, 713,078,155 bytes, L5=9 L6=9
      file-name   level     file-size  smallest-key largest-key
      000017.sst  L5       39,646,656  0002800001   0003112000
      000018.sst  L5       39,714,036  0003112001   0003424000
      000019.sst  L5       39,733,807  0003424001   0003736000
      000020.sst  L5       39,634,266  0003736001   0004048000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000022.sst  L5       39,651,458  0004360001   0004672000
      000023.sst  L5       39,701,916  0004672001   0004984000
      000024.sst  L5       39,611,404  0004984001   0005296000
      000025.sst  L5       39,653,012  0005296001   0005608000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 049  t=  4.794s] COMPACTION DONE   job=25 compaction #6 L0 -> L5 TRIVIAL MOVE (no rewrite): 4 in, 4 out, read 0,
wrote 0 bytes
  SSTables after ('+' = output): 18 files, 713,078,155 bytes, L5=9 L6=9
      file-name   level     file-size  smallest-key largest-key
      000017.sst  L5       39,646,656  0002800001   0003112000
      000018.sst  L5       39,714,036  0003112001   0003424000
      000019.sst  L5       39,733,807  0003424001   0003736000
      000020.sst  L5       39,634,266  0003736001   0004048000
      000021.sst  L5       39,716,518  0004048001   0004360000
    + 000022.sst  L5       39,651,458  0004360001   0004672000
    + 000023.sst  L5       39,701,916  0004672001   0004984000
    + 000024.sst  L5       39,611,404  0004984001   0005296000
    + 000025.sst  L5       39,653,012  0005296001   0005608000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 050  t=  4.794s] COMPACTION BEGIN  job=27 reason=kLevelMaxLevelSize L5 -> L6, 4 input files
  SSTables before ('*' = input): 18 files, 713,078,155 bytes, L5=9 L6=9
      file-name   level     file-size  smallest-key largest-key
    * 000017.sst  L5       39,646,656  0002800001   0003112000
    * 000018.sst  L5       39,714,036  0003112001   0003424000
    * 000019.sst  L5       39,733,807  0003424001   0003736000
    * 000020.sst  L5       39,634,266  0003736001   0004048000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000022.sst  L5       39,651,458  0004360001   0004672000
      000023.sst  L5       39,701,916  0004672001   0004984000
      000024.sst  L5       39,611,404  0004984001   0005296000
      000025.sst  L5       39,653,012  0005296001   0005608000
```

```
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000

[event 051  t=  4.798s] COMPACTION DONE   job=27 compaction #7 L5 -> L6 TRIVIAL MOVE (no rewrite): 4 in, 4 out, read 0,
wrote 0 bytes
  SSTables after ('+' = output): 18 files, 713,078,155 bytes, L5=5 L6=13
      file-name   level     file-size  smallest-key largest-key
      000021.sst  L5       39,716,518  0004048001   0004360000
      000022.sst  L5       39,651,458  0004360001   0004672000
      000023.sst  L5       39,701,916  0004672001   0004984000
      000024.sst  L5       39,611,404  0004984001   0005296000
      000025.sst  L5       39,653,012  0005296001   0005608000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000
    + 000017.sst  L6       39,646,656  0002800001   0003112000
    + 000018.sst  L6       39,714,036  0003112001   0003424000
    + 000019.sst  L6       39,733,807  0003424001   0003736000
    + 000020.sst  L6       39,634,266  0003736001   0004048000

[event 052  t=  5.008s] FLUSH COMPLETED   job=26 flush #19 wrote 000026.sst (39,754,692 bytes) to L0, seqno 1,402,001..
1,480,000
  SSTables after: 19 files, 752,832,847 bytes, L0=1 L5=5 L6=13
      file-name   level     file-size  smallest-key largest-key
    + 000026.sst  L0       39,754,692  0005608001   0005920000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000022.sst  L5       39,651,458  0004360001   0004672000
      000023.sst  L5       39,701,916  0004672001   0004984000
      000024.sst  L5       39,611,404  0004984001   0005296000
      000025.sst  L5       39,653,012  0005296001   0005608000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000
      000017.sst  L6       39,646,656  0002800001   0003112000
      000018.sst  L6       39,714,036  0003112001   0003424000
      000019.sst  L6       39,733,807  0003424001   0003736000
      000020.sst  L6       39,634,266  0003736001   0004048000

[event 053  t=  5.009s] FLUSH BEGIN       job=28 reason=MANUAL_FLUSH
  SSTables before: 19 files, 752,832,847 bytes, L0=1 L5=5 L6=13
      file-name   level     file-size  smallest-key largest-key
      000026.sst  L0       39,754,692  0005608001   0005920000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000022.sst  L5       39,651,458  0004360001   0004672000
      000023.sst  L5       39,701,916  0004672001   0004984000
      000024.sst  L5       39,611,404  0004984001   0005296000
      000025.sst  L5       39,653,012  0005296001   0005608000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000
      000017.sst  L6       39,646,656  0002800001   0003112000
      000018.sst  L6       39,714,036  0003112001   0003424000
      000019.sst  L6       39,733,807  0003424001   0003736000
      000020.sst  L6       39,634,266  0003736001   0004048000

[event 054  t=  5.066s] FLUSH COMPLETED   job=28 flush #20 wrote 000027.sst (10,187,938 bytes) to L0, seqno 1,480,001..
1,500,000
  SSTables after: 20 files, 763,020,785 bytes, L0=2 L5=5 L6=13
      file-name   level     file-size  smallest-key largest-key
      000026.sst  L0       39,754,692  0005608001   0005920000
```

```
    + 000027.sst  L0       10,187,938  0005920001   0006000000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000022.sst  L5       39,651,458  0004360001   0004672000
      000023.sst  L5       39,701,916  0004672001   0004984000
      000024.sst  L5       39,611,404  0004984001   0005296000
      000025.sst  L5       39,653,012  0005296001   0005608000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000
      000017.sst  L6       39,646,656  0002800001   0003112000
      000018.sst  L6       39,714,036  0003112001   0003424000
      000019.sst  L6       39,733,807  0003424001   0003736000
      000020.sst  L6       39,634,266  0003736001   0004048000

======== LEVELED totals ========
orders loaded         : 1,500,000
flushes               : 20 (763,020,785 bytes written to L0)
compactions           : 7 (7 trivial moves, 0 merges)
compaction read bytes : 0
compaction write bytes: 0
write amplification   : 1.00  ((flush + compaction writes) / flush writes)
```

6.3 Leveled: final SSTables, table properties and index excerpt (`output/leveled/final_sstables.txt`)

```
======== LEVELED final SSTables (/Users/avaish/IE-494/lab9/db/leveled/CORDERS) ========
orders loaded 1,500,000, load 4.755 s, flush + settle 1.261 s, db directory 763,033,722 bytes
  SSTables final: 20 files, 763,020,785 bytes, L0=2 L5=5 L6=13
      file-name   level     file-size  smallest-key largest-key
      000026.sst  L0       39,754,692  0005608001   0005920000
      000027.sst  L0       10,187,938  0005920001   0006000000
      000021.sst  L5       39,716,518  0004048001   0004360000
      000022.sst  L5       39,651,458  0004360001   0004672000
      000023.sst  L5       39,701,916  0004672001   0004984000
      000024.sst  L5       39,611,404  0004984001   0005296000
      000025.sst  L5       39,653,012  0005296001   0005608000
      000008.sst  L6       39,646,772  0000000001   0000312000
      000009.sst  L6       39,719,410  0000312001   0000624000
      000010.sst  L6       39,670,846  0000624001   0000936000
      000011.sst  L6       39,622,861  0000936001   0001248000
      000012.sst  L6       39,637,956  0001248001   0001560000
      000013.sst  L6       38,771,506  0001560001   0001864000
      000014.sst  L6       39,630,369  0001864001   0002176000
      000015.sst  L6       39,669,707  0002176001   0002488000
      000016.sst  L6       39,645,655  0002488001   0002800000
      000017.sst  L6       39,646,656  0002800001   0003112000
      000018.sst  L6       39,714,036  0003112001   0003424000
      000019.sst  L6       39,733,807  0003424001   0003736000
      000020.sst  L6       39,634,266  0003736001   0004048000

Table properties and index block of each SSTable (first/last 3 index entries; all entries are in index_dump.txt):

  000026.sst  L0  39,754,692 bytes  keys 0005608001 .. 0005920000
    entries 78,000, data blocks 2,103, data 39,705,773 B, index 47,771 B, raw key 1,404,000 B, raw value 134,535,488 B,
LZ4, format_version 5
    index block @ 39,705,773 (47,766 B), 2,103 entries, 2103 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0005608131                 0     18,977
            2  000560829             18,982     19,098
            3  0005608420            38,085     18,310
          ...  (2,097 more entries)
         2101  00059197           39,650,759     18,739
         2102  0005919846         39,669,503     18,866
         2103  0005920000         39,688,374     17,394

  000027.sst  L0  10,187,938 bytes  keys 0005920001 .. 0006000000
    entries 20,000, data blocks 540, data 10,174,586 B, index 12,205 B, raw key 360,000 B, raw value 34,465,440 B, LZ4,
format_version 5
    index block @ 10,174,586 (12,200 B), 540 entries, 540 restart points, user keys, delta-encoded handles; check OK (1
entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0005920163                 0     19,153
            2  0005920291             19,158     18,956
            3  0005920448             38,119     19,057
```

```
          ...  (534 more entries)
          538  0005999781         10,131,488     18,521
          539  0005999942         10,150,014     19,282
          540  0006000000         10,169,301      5,280

  000021.sst  L5  39,716,518 bytes  keys 0004048001 .. 0004360000
    entries 78,000, data blocks 2,102, data 39,667,628 B, index 47,742 B, raw key 1,404,000 B, raw value 134,395,344 B,
LZ4, format_version 5
    index block @ 39,667,628 (47,737 B), 2,102 entries, 2102 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0004048132                 0     18,872
            2  0004048292            18,877     18,733
            3  000404842             37,615     18,760
          ...  (2,096 more entries)
         2100  0004359781         39,622,780     18,941
         2101  0004359938         39,641,726     19,030
         2102  0004360000         39,660,761      6,862

  000022.sst  L5  39,651,458 bytes  keys 0004360001 .. 0004672000
    entries 78,000, data blocks 2,098, data 39,602,655 B, index 47,655 B, raw key 1,404,000 B, raw value 134,179,104 B,
LZ4, format_version 5
    index block @ 39,602,655 (47,650 B), 2,098 entries, 2098 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  000436014                  0     18,919
            2  0004360292            18,924     18,856
            3  0004360450            37,785     18,858
          ...  (2,092 more entries)
         2096  000467176          39,553,182     18,904
         2097  0004671908         39,572,091     19,077
         2098  0004672000         39,591,173     11,477

  000023.sst  L5  39,701,916 bytes  keys 0004672001 .. 0004984000
    entries 78,000, data blocks 2,100, data 39,653,127 B, index 47,641 B, raw key 1,404,000 B, raw value 134,366,710 B,
LZ4, format_version 5
    index block @ 39,653,127 (47,636 B), 2,100 entries, 2100 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  000467214                  0     19,094
            2  0004672292            19,099     18,871
            3  0004672452            37,975     18,823
          ...  (2,094 more entries)
         2098  0004983712         39,597,617     18,846
         2099  0004983846         39,616,468     18,812
         2100  0004984000         39,635,285     17,837

  000024.sst  L5  39,611,404 bytes  keys 0004984001 .. 0005296000
    entries 78,000, data blocks 2,095, data 39,562,696 B, index 47,560 B, raw key 1,404,000 B, raw value 133,973,238 B,
LZ4, format_version 5
    index block @ 39,562,696 (47,555 B), 2,095 entries, 2095 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0004984131                 0     18,967
            2  0004984292            18,972     19,004
            3  000498443             37,981     19,132
          ...  (2,089 more entries)
         2093  0005295810         39,518,981     19,095
         2094  0005295941         39,538,081     18,816
         2095  0005296000         39,556,902      5,789

  000025.sst  L5  39,653,012 bytes  keys 0005296001 .. 0005608000
    entries 78,000, data blocks 2,098, data 39,604,225 B, index 47,639 B, raw key 1,404,000 B, raw value 134,183,525 B,
LZ4, format_version 5
    index block @ 39,604,225 (47,634 B), 2,098 entries, 2098 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  000529614                  0     18,932
            2  00052963              18,937     19,350
            3  0005296451            38,292     18,985
          ...  (2,092 more entries)
         2096  0005607685         39,549,346     19,250
         2097  0005607845         39,568,601     19,039
         2098  0005608000         39,587,645     16,575

  000008.sst  L6  39,646,772 bytes  keys 0000000001 .. 0000312000
    entries 78,000, data blocks 2,090, data 39,598,168 B, index 47,458 B, raw key 1,404,000 B, raw value 133,661,192 B,
LZ4, format_version 5
    index block @ 39,598,168 (47,453 B), 2,090 entries, 2090 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0000000134                 0     18,971
            2  000000029             18,976     19,090
            3  000000045             38,071     19,033
          ...  (2,084 more entries)
```

```
         2088  0000311776         39,549,891     19,074
         2089  0000311908         39,568,970     18,671
         2090  0000312000         39,587,646     10,517

  000009.sst  L6  39,719,410 bytes  keys 0000312001 .. 0000624000
    entries 78,000, data blocks 2,096, data 39,670,671 B, index 47,591 B, raw key 1,404,000 B, raw value 134,047,204 B,
LZ4, format_version 5
    index block @ 39,670,671 (47,586 B), 2,096 entries, 2096 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0000312134                 0     18,867
            2  0000312291            18,872     18,995
            3  000031243             37,872     18,927
          ...  (2,090 more entries)
         2094  0000623683         39,614,241     18,898
         2095  0000623844         39,633,144     19,167
         2096  0000624000         39,652,316     18,350

  000010.sst  L6  39,670,846 bytes  keys 0000624001 .. 0000936000
    entries 78,000, data blocks 2,092, data 39,622,242 B, index 47,456 B, raw key 1,404,000 B, raw value 133,844,372 B,
LZ4, format_version 5
    index block @ 39,622,242 (47,451 B), 2,092 entries, 2092 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0000624133                 0     18,657
            2  0000624262            18,662     18,759
            3  0000624416            37,426     18,869
          ...  (2,086 more entries)
         2090  0000935716         39,567,782     18,595
         2091  0000935846         39,586,382     18,995
         2092  0000936000         39,605,382     16,855

  000011.sst  L6  39,622,861 bytes  keys 0000936001 .. 0001248000
    entries 78,000, data blocks 2,095, data 39,574,134 B, index 47,579 B, raw key 1,404,000 B, raw value 133,962,842 B,
LZ4, format_version 5
    index block @ 39,574,134 (47,574 B), 2,095 entries, 2095 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0000936128                 0     18,525
            2  0000936291            18,530     19,431
            3  000093643             37,966     19,034
          ...  (2,089 more entries)
         2093  0001247717         39,523,803     18,672
         2094  0001247876         39,542,480     18,955
         2095  0001248000         39,561,440     12,689

  000012.sst  L6  39,637,956 bytes  keys 0001248001 .. 0001560000
    entries 78,000, data blocks 2,097, data 39,589,197 B, index 47,611 B, raw key 1,404,000 B, raw value 134,117,858 B,
LZ4, format_version 5
    index block @ 39,589,197 (47,606 B), 2,097 entries, 2097 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0001248128                 0     18,576
            2  0001248257            18,581     19,170
            3  000124842             37,756     19,149
          ...  (2,091 more entries)
         2095  0001559844         39,548,629     19,200
         2096  0001559973         39,567,834     18,589
         2097  0001560000         39,586,428      2,764

  000013.sst  L6  38,771,506 bytes  keys 0001560001 .. 0001864000
    entries 76,000, data blocks 2,052, data 38,723,769 B, index 46,589 B, raw key 1,368,000 B, raw value 131,196,487 B,
LZ4, format_version 5
    index block @ 38,723,769 (46,584 B), 2,052 entries, 2052 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0001560132                 0     19,027
            2  0001560290            19,032     19,335
            3  000156043             38,372     18,572
          ...  (2,046 more entries)
         2050  0001863781         38,679,004     18,920
         2051  0001863940         38,697,929     18,952
         2052  0001864000         38,716,886      6,878

  000014.sst  L6  39,630,369 bytes  keys 0001864001 .. 0002176000
    entries 78,000, data blocks 2,096, data 39,581,636 B, index 47,585 B, raw key 1,404,000 B, raw value 134,102,151 B,
LZ4, format_version 5
    index block @ 39,581,636 (47,580 B), 2,096 entries, 2096 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0001864193                 0     19,312
            2  0001864352            19,317     18,466
            3  0001864484            37,788     18,934
          ...  (2,090 more entries)
         2094  0002175714         39,526,348     18,690
```

```
         2095  000217585          39,545,043     19,087
         2096  0002176000         39,564,135     17,496

  000015.sst  L6  39,669,707 bytes  keys 0002176001 .. 0002488000
    entries 78,000, data blocks 2,099, data 39,620,889 B, index 47,670 B, raw key 1,404,000 B, raw value 134,222,913 B,
LZ4, format_version 5
    index block @ 39,620,889 (47,665 B), 2,099 entries, 2099 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0002176130                 0     18,356
            2  0002176260            18,361     18,856
            3  0002176420            37,222     19,181
          ...  (2,093 more entries)
         2097  0002487781         39,574,773     18,944
         2098  0002487938         39,593,722     18,898
         2099  0002488000         39,612,625      8,259

  000016.sst  L6  39,645,655 bytes  keys 0002488001 .. 0002800000
    entries 78,000, data blocks 2,097, data 39,596,833 B, index 47,674 B, raw key 1,404,000 B, raw value 134,144,233 B,
LZ4, format_version 5
    index block @ 39,596,833 (47,669 B), 2,097 entries, 2097 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0002488160                 0     18,703
            2  0002488291            18,708     19,010
            3  000248843             37,723     18,908
          ...  (2,091 more entries)
         2095  0002799778         39,550,476     18,752
         2096  0002799936         39,569,233     18,930
         2097  0002800000         39,588,168      8,660

  000017.sst  L6  39,646,656 bytes  keys 0002800001 .. 0003112000
    entries 78,000, data blocks 2,098, data 39,597,875 B, index 47,633 B, raw key 1,404,000 B, raw value 134,160,846 B,
LZ4, format_version 5
    index block @ 39,597,875 (47,628 B), 2,098 entries, 2098 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0002800132                 0     18,904
            2  000280029             18,909     18,877
            3  0002800420            37,791     19,052
          ...  (2,092 more entries)
         2096  0003111683         39,541,736     18,765
         2097  0003111844         39,560,506     18,722
         2098  0003112000         39,579,233     18,637

  000018.sst  L6  39,714,036 bytes  keys 0003112001 .. 0003424000
    entries 78,000, data blocks 2,102, data 39,665,154 B, index 47,734 B, raw key 1,404,000 B, raw value 134,436,982 B,
LZ4, format_version 5
    index block @ 39,665,154 (47,729 B), 2,102 entries, 2102 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0003112134                 0     19,111
            2  0003112321            19,116     19,177
            3  0003112453            38,298     18,559
          ...  (2,096 more entries)
         2100  0003423682         39,610,190     19,044
         2101  0003423842         39,629,239     19,214
         2102  0003424000         39,648,458     16,691

  000019.sst  L6  39,733,807 bytes  keys 0003424001 .. 0003736000
    entries 78,000, data blocks 2,102, data 39,684,919 B, index 47,740 B, raw key 1,404,000 B, raw value 134,474,268 B,
LZ4, format_version 5
    index block @ 39,684,919 (47,735 B), 2,102 entries, 2102 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0003424133                 0     18,938
            2  0003424292            18,943     18,740
            3  0003424448            37,688     18,833
          ...  (2,096 more entries)
         2100  0003735750         39,635,320     19,023
         2101  0003735905         39,654,348     18,888
         2102  0003736000         39,673,241     11,673

  000020.sst  L6  39,634,266 bytes  keys 0003736001 .. 0004048000
    entries 78,000, data blocks 2,097, data 39,585,520 B, index 47,598 B, raw key 1,404,000 B, raw value 134,133,206 B,
LZ4, format_version 5
    index block @ 39,585,520 (47,593 B), 2,097 entries, 2097 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  000373614                  0     18,463
            2  0003736293            18,468     19,010
            3  0003736451            37,483     19,149
          ...  (2,091 more entries)
         2095  0004047750         39,537,287     18,794
```

```
         2096  0004047908         39,556,086     18,918
         2097  0004048000         39,575,009     10,506
```

6.4 Universal: every flush and compaction (`output/universal/events.txt`)

```
UNIVERSAL compaction: every flush and compaction while loading corders.json
rocksdb path: /Users/avaish/IE-494/lab9/db/universal/CORDERS
options: compaction_style=kCompactionStyleUniversal (size_ratio=1, min_merge_width=2, max_size_amplification_percent=20
0), write_buffer_size=128 MiB x 2, level0_file_num_compaction_trigger=4, target_file_size_base=64 MiB, max_bytes_for_le
vel_base=256 MiB, compression=LZ4, block_size=64 KiB, format_version=5, index compression off, max_background_jobs=2, W
AL off, batch=2,000
'*' marks compaction inputs, '+' marks files written (or moved) by the job.

[event 001  t=  0.269s] FLUSH BEGIN       job=2 reason=WRITE_BUFFER_FULL
  SSTables before: 0 files, 0 bytes, no levels

[event 002  t=  0.446s] FLUSH COMPLETED   job=2 flush #1 wrote 000008.sst (39,646,772 bytes) to L0, seqno 1..78,000
  SSTables after: 1 files, 39,646,772 bytes, L0=1
      file-name   level     file-size  smallest-key largest-key
    + 000008.sst  L0       39,646,772  0000000001   0000312000

[event 003  t=  0.446s] FLUSH BEGIN       job=3 reason=WRITE_BUFFER_FULL
  SSTables before: 1 files, 39,646,772 bytes, L0=1
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000

[event 004  t=  0.609s] FLUSH COMPLETED   job=3 flush #2 wrote 000009.sst (39,719,410 bytes) to L0, seqno 78,001..156,0
00
  SSTables after: 2 files, 79,366,182 bytes, L0=2
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
    + 000009.sst  L0       39,719,410  0000312001   0000624000

[event 005  t=  0.609s] FLUSH BEGIN       job=4 reason=WRITE_BUFFER_FULL
  SSTables before: 2 files, 79,366,182 bytes, L0=2
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000

[event 006  t=  0.790s] FLUSH COMPLETED   job=4 flush #3 wrote 000010.sst (39,670,846 bytes) to L0, seqno 156,001..234,
000
  SSTables after: 3 files, 119,037,028 bytes, L0=3
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
    + 000010.sst  L0       39,670,846  0000624001   0000936000

[event 007  t=  0.790s] FLUSH BEGIN       job=5 reason=WRITE_BUFFER_FULL
  SSTables before: 3 files, 119,037,028 bytes, L0=3
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000

[event 008  t=  0.959s] FLUSH COMPLETED   job=5 flush #4 wrote 000011.sst (39,622,861 bytes) to L0, seqno 234,001..312,
000
  SSTables after: 4 files, 158,659,889 bytes, L0=4
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
    + 000011.sst  L0       39,622,861  0000936001   0001248000

[event 009  t=  0.959s] COMPACTION BEGIN  job=6 reason=kUniversalSizeAmplification L0 -> L6, 4 input files
  SSTables before ('*' = input): 4 files, 158,659,889 bytes, L0=4
      file-name   level     file-size  smallest-key largest-key
    * 000008.sst  L0       39,646,772  0000000001   0000312000
    * 000009.sst  L0       39,719,410  0000312001   0000624000
    * 000010.sst  L0       39,670,846  0000624001   0000936000
    * 000011.sst  L0       39,622,861  0000936001   0001248000

[event 010  t=  0.960s] FLUSH BEGIN       job=7 reason=WRITE_BUFFER_FULL
  SSTables before: 4 files, 158,659,889 bytes, L0=4
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000

[event 011  t=  1.170s] FLUSH COMPLETED   job=7 flush #5 wrote 000012.sst (39,637,956 bytes) to L0, seqno 312,001..390,
000
  SSTables after: 5 files, 198,297,845 bytes, L0=5
```

```
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000
    + 000012.sst  L0       39,637,956  0001248001   0001560000

[event 012  t=  1.170s] FLUSH BEGIN       job=8 reason=WRITE_BUFFER_FULL
  SSTables before: 5 files, 198,297,845 bytes, L0=5
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000
      000012.sst  L0       39,637,956  0001248001   0001560000

[event 013  t=  1.487s] FLUSH COMPLETED   job=8 flush #6 wrote 000014.sst (38,771,506 bytes) to L0, seqno 390,001..466,
000
  SSTables after: 6 files, 237,069,351 bytes, L0=6
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000
      000012.sst  L0       39,637,956  0001248001   0001560000
    + 000014.sst  L0       38,771,506  0001560001   0001864000

[event 014  t=  1.487s] FLUSH BEGIN       job=9 reason=WRITE_BUFFER_FULL
  SSTables before: 6 files, 237,069,351 bytes, L0=6
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000

[event 015  t=  1.865s] FLUSH COMPLETED   job=9 flush #7 wrote 000015.sst (39,630,369 bytes) to L0, seqno 466,001..544,
000
  SSTables after: 7 files, 276,699,720 bytes, L0=7
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
    + 000015.sst  L0       39,630,369  0001864001   0002176000

[event 016  t=  1.866s] FLUSH BEGIN       job=10 reason=WRITE_BUFFER_FULL
  SSTables before: 7 files, 276,699,720 bytes, L0=7
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000

[event 017  t=  2.259s] FLUSH COMPLETED   job=10 flush #8 wrote 000017.sst (39,669,707 bytes) to L0, seqno 544,001..62
2,000
  SSTables after: 8 files, 316,369,427 bytes, L0=8
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
    + 000017.sst  L0       39,669,707  0002176001   0002488000

[event 018  t=  2.261s] FLUSH BEGIN       job=11 reason=WRITE_BUFFER_FULL
  SSTables before: 8 files, 316,369,427 bytes, L0=8
      file-name   level     file-size  smallest-key largest-key
      000008.sst  L0       39,646,772  0000000001   0000312000
      000009.sst  L0       39,719,410  0000312001   0000624000
      000010.sst  L0       39,670,846  0000624001   0000936000
      000011.sst  L0       39,622,861  0000936001   0001248000
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
```

```
[event 019  t=  2.537s] COMPACTION DONE   job=6 compaction #1 L0 -> L6 merge: 4 in, 3 out, read 158,659,889, wrote 158,
153,336 bytes
  SSTables after ('+' = output): 7 files, 315,862,874 bytes, L0=4 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
    + 000013.sst  L6       67,204,013  0000000001   0000529860
    + 000016.sst  L6       67,201,977  0000529861   0001060352
    + 000019.sst  L6       23,747,346  0001060353   0001248000

[event 020  t=  2.538s] COMPACTION BEGIN  job=12 reason=kUniversalSizeRatio L0 -> L6, 7 input files
  SSTables before ('*' = input): 7 files, 315,862,874 bytes, L0=4 L6=3
      file-name   level     file-size  smallest-key largest-key
    * 000012.sst  L0       39,637,956  0001248001   0001560000
    * 000014.sst  L0       38,771,506  0001560001   0001864000
    * 000015.sst  L0       39,630,369  0001864001   0002176000
    * 000017.sst  L0       39,669,707  0002176001   0002488000
    * 000013.sst  L6       67,204,013  0000000001   0000529860
    * 000016.sst  L6       67,201,977  0000529861   0001060352
    * 000019.sst  L6       23,747,346  0001060353   0001248000

[event 021  t=  2.589s] FLUSH COMPLETED   job=11 flush #9 wrote 000018.sst (39,645,655 bytes) to L0, seqno 622,001..70
0,000
  SSTables after: 8 files, 355,508,529 bytes, L0=5 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
    + 000018.sst  L0       39,645,655  0002488001   0002800000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 022  t=  2.596s] FLUSH BEGIN       job=13 reason=WRITE_BUFFER_FULL
  SSTables before: 8 files, 355,508,529 bytes, L0=5 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 023  t=  2.953s] FLUSH COMPLETED   job=13 flush #10 wrote 000021.sst (39,646,656 bytes) to L0, seqno 700,001..77
8,000
  SSTables after: 9 files, 395,155,185 bytes, L0=6 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
    + 000021.sst  L0       39,646,656  0002800001   0003112000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 024  t=  2.954s] FLUSH BEGIN       job=14 reason=WRITE_BUFFER_FULL
  SSTables before: 9 files, 395,155,185 bytes, L0=6 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 025  t=  3.362s] FLUSH COMPLETED   job=14 flush #11 wrote 000022.sst (39,714,036 bytes) to L0, seqno 778,001..85
6,000
  SSTables after: 10 files, 434,869,221 bytes, L0=7 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
```

```
    + 000022.sst  L0       39,714,036  0003112001   0003424000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 026  t=  3.363s] FLUSH BEGIN       job=15 reason=WRITE_BUFFER_FULL
  SSTables before: 10 files, 434,869,221 bytes, L0=7 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 027  t=  4.404s] FLUSH COMPLETED   job=15 flush #12 wrote 000023.sst (39,733,807 bytes) to L0, seqno 856,001..93
4,000
  SSTables after: 11 files, 474,603,028 bytes, L0=8 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
    + 000023.sst  L0       39,733,807  0003424001   0003736000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 028  t=  4.407s] FLUSH BEGIN       job=16 reason=WRITE_BUFFER_FULL
  SSTables before: 11 files, 474,603,028 bytes, L0=8 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 029  t=  4.732s] FLUSH COMPLETED   job=16 flush #13 wrote 000025.sst (39,634,266 bytes) to L0, seqno 934,001..1,
012,000
  SSTables after: 12 files, 514,237,294 bytes, L0=9 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
    + 000025.sst  L0       39,634,266  0003736001   0004048000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 030  t=  4.732s] FLUSH BEGIN       job=17 reason=WRITE_BUFFER_FULL
  SSTables before: 12 files, 514,237,294 bytes, L0=9 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 031  t=  5.113s] FLUSH COMPLETED   job=17 flush #14 wrote 000026.sst (39,716,518 bytes) to L0, seqno 1,012,001..
1,090,000
```

```
  SSTables after: 13 files, 553,953,812 bytes, L0=10 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
    + 000026.sst  L0       39,716,518  0004048001   0004360000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 032  t=  5.115s] FLUSH BEGIN       job=18 reason=WRITE_BUFFER_FULL
  SSTables before: 13 files, 553,953,812 bytes, L0=10 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 033  t=  5.439s] FLUSH COMPLETED   job=18 flush #15 wrote 000028.sst (39,651,458 bytes) to L0, seqno 1,090,001..
1,168,000
  SSTables after: 14 files, 593,605,270 bytes, L0=11 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
    + 000028.sst  L0       39,651,458  0004360001   0004672000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 034  t=  5.439s] FLUSH BEGIN       job=19 reason=WRITE_BUFFER_FULL
  SSTables before: 14 files, 593,605,270 bytes, L0=11 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
      000028.sst  L0       39,651,458  0004360001   0004672000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 035  t=  5.827s] FLUSH COMPLETED   job=19 flush #16 wrote 000029.sst (39,701,916 bytes) to L0, seqno 1,168,001..
1,246,000
  SSTables after: 15 files, 633,307,186 bytes, L0=12 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
      000028.sst  L0       39,651,458  0004360001   0004672000
```

```
    + 000029.sst  L0       39,701,916  0004672001   0004984000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 036  t=  5.829s] FLUSH BEGIN       job=20 reason=WRITE_BUFFER_FULL
  SSTables before: 15 files, 633,307,186 bytes, L0=12 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
      000028.sst  L0       39,651,458  0004360001   0004672000
      000029.sst  L0       39,701,916  0004672001   0004984000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 037  t=  6.198s] FLUSH COMPLETED   job=20 flush #17 wrote 000031.sst (39,611,404 bytes) to L0, seqno 1,246,001..
1,324,000
  SSTables after: 16 files, 672,918,590 bytes, L0=13 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
      000028.sst  L0       39,651,458  0004360001   0004672000
      000029.sst  L0       39,701,916  0004672001   0004984000
    + 000031.sst  L0       39,611,404  0004984001   0005296000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 038  t=  6.199s] FLUSH BEGIN       job=21 reason=WRITE_BUFFER_FULL
  SSTables before: 16 files, 672,918,590 bytes, L0=13 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
      000028.sst  L0       39,651,458  0004360001   0004672000
      000029.sst  L0       39,701,916  0004672001   0004984000
      000031.sst  L0       39,611,404  0004984001   0005296000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 039  t=  6.511s] FLUSH COMPLETED   job=21 flush #18 wrote 000032.sst (39,653,012 bytes) to L0, seqno 1,324,001..
1,402,000
  SSTables after: 17 files, 712,571,602 bytes, L0=14 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
      000028.sst  L0       39,651,458  0004360001   0004672000
      000029.sst  L0       39,701,916  0004672001   0004984000
      000031.sst  L0       39,611,404  0004984001   0005296000
    + 000032.sst  L0       39,653,012  0005296001   0005608000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
```

```
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 040  t=  6.512s] FLUSH BEGIN       job=22 reason=WRITE_BUFFER_FULL
  SSTables before: 17 files, 712,571,602 bytes, L0=14 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
      000028.sst  L0       39,651,458  0004360001   0004672000
      000029.sst  L0       39,701,916  0004672001   0004984000
      000031.sst  L0       39,611,404  0004984001   0005296000
      000032.sst  L0       39,653,012  0005296001   0005608000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 041  t=  6.884s] FLUSH COMPLETED   job=22 flush #19 wrote 000034.sst (39,754,692 bytes) to L0, seqno 1,402,001..
1,480,000
  SSTables after: 18 files, 752,326,294 bytes, L0=15 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
      000028.sst  L0       39,651,458  0004360001   0004672000
      000029.sst  L0       39,701,916  0004672001   0004984000
      000031.sst  L0       39,611,404  0004984001   0005296000
      000032.sst  L0       39,653,012  0005296001   0005608000
    + 000034.sst  L0       39,754,692  0005608001   0005920000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 042  t=  6.886s] FLUSH BEGIN       job=23 reason=MANUAL_FLUSH
  SSTables before: 18 files, 752,326,294 bytes, L0=15 L6=3
      file-name   level     file-size  smallest-key largest-key
      000012.sst  L0       39,637,956  0001248001   0001560000
      000014.sst  L0       38,771,506  0001560001   0001864000
      000015.sst  L0       39,630,369  0001864001   0002176000
      000017.sst  L0       39,669,707  0002176001   0002488000
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
      000028.sst  L0       39,651,458  0004360001   0004672000
      000029.sst  L0       39,701,916  0004672001   0004984000
      000031.sst  L0       39,611,404  0004984001   0005296000
      000032.sst  L0       39,653,012  0005296001   0005608000
      000034.sst  L0       39,754,692  0005608001   0005920000
      000013.sst  L6       67,204,013  0000000001   0000529860
      000016.sst  L6       67,201,977  0000529861   0001060352
      000019.sst  L6       23,747,346  0001060353   0001248000

[event 043  t=  6.926s] COMPACTION DONE   job=12 compaction #2 L0 -> L6 merge: 7 in, 5 out, read 315,862,874, wrote 31
5,364,104 bytes
  SSTables after ('+' = output): 16 files, 751,827,524 bytes, L0=11 L6=5
      file-name   level     file-size  smallest-key largest-key
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
      000028.sst  L0       39,651,458  0004360001   0004672000
      000029.sst  L0       39,701,916  0004672001   0004984000
      000031.sst  L0       39,611,404  0004984001   0005296000
      000032.sst  L0       39,653,012  0005296001   0005608000
      000034.sst  L0       39,754,692  0005608001   0005920000
    + 000020.sst  L6       67,204,013  0000000001   0000529860
    + 000024.sst  L6       67,201,977  0000529861   0001060352
```

```
    + 000027.sst  L6       67,192,231  0001060353   0001590949
    + 000030.sst  L6       67,194,260  0001590950   0002120322
    + 000033.sst  L6       46,571,623  0002120323   0002488000

[event 044  t=  6.927s] COMPACTION BEGIN  job=24 reason=kUniversalSizeRatio L0 -> L6, 16 input files
  SSTables before ('*' = input): 16 files, 751,827,524 bytes, L0=11 L6=5
      file-name   level     file-size  smallest-key largest-key
    * 000018.sst  L0       39,645,655  0002488001   0002800000
    * 000021.sst  L0       39,646,656  0002800001   0003112000
    * 000022.sst  L0       39,714,036  0003112001   0003424000
    * 000023.sst  L0       39,733,807  0003424001   0003736000
    * 000025.sst  L0       39,634,266  0003736001   0004048000
    * 000026.sst  L0       39,716,518  0004048001   0004360000
    * 000028.sst  L0       39,651,458  0004360001   0004672000
    * 000029.sst  L0       39,701,916  0004672001   0004984000
    * 000031.sst  L0       39,611,404  0004984001   0005296000
    * 000032.sst  L0       39,653,012  0005296001   0005608000
    * 000034.sst  L0       39,754,692  0005608001   0005920000
    * 000020.sst  L6       67,204,013  0000000001   0000529860
    * 000024.sst  L6       67,201,977  0000529861   0001060352
    * 000027.sst  L6       67,192,231  0001060353   0001590949
    * 000030.sst  L6       67,194,260  0001590950   0002120322
    * 000033.sst  L6       46,571,623  0002120323   0002488000

[event 045  t=  7.005s] FLUSH COMPLETED   job=23 flush #20 wrote 000035.sst (10,187,938 bytes) to L0, seqno 1,480,001..
1,500,000
  SSTables after: 17 files, 762,015,462 bytes, L0=12 L6=5
      file-name   level     file-size  smallest-key largest-key
      000018.sst  L0       39,645,655  0002488001   0002800000
      000021.sst  L0       39,646,656  0002800001   0003112000
      000022.sst  L0       39,714,036  0003112001   0003424000
      000023.sst  L0       39,733,807  0003424001   0003736000
      000025.sst  L0       39,634,266  0003736001   0004048000
      000026.sst  L0       39,716,518  0004048001   0004360000
      000028.sst  L0       39,651,458  0004360001   0004672000
      000029.sst  L0       39,701,916  0004672001   0004984000
      000031.sst  L0       39,611,404  0004984001   0005296000
      000032.sst  L0       39,653,012  0005296001   0005608000
      000034.sst  L0       39,754,692  0005608001   0005920000
    + 000035.sst  L0       10,187,938  0005920001   0006000000
      000020.sst  L6       67,204,013  0000000001   0000529860
      000024.sst  L6       67,201,977  0000529861   0001060352
      000027.sst  L6       67,192,231  0001060353   0001590949
      000030.sst  L6       67,194,260  0001590950   0002120322
      000033.sst  L6       46,571,623  0002120323   0002488000

[event 046  t= 11.568s] COMPACTION DONE   job=24 compaction #3 L0 -> L6 merge: 16 in, 12 out, read 751,827,524, wrote 7
50,433,081 bytes
  SSTables after ('+' = output): 13 files, 760,621,019 bytes, L0=1 L6=12
      file-name   level     file-size  smallest-key largest-key
      000035.sst  L0       10,187,938  0005920001   0006000000
    + 000036.sst  L6       67,204,013  0000000001   0000529860
    + 000037.sst  L6       67,201,977  0000529861   0001060352
    + 000038.sst  L6       67,192,231  0001060353   0001590949
    + 000039.sst  L6       67,194,260  0001590950   0002120322
    + 000040.sst  L6       67,206,723  0002120323   0002651335
    + 000041.sst  L6       67,200,041  0002651360   0003181378
    + 000042.sst  L6       67,205,695  0003181379   0003710595
    + 000043.sst  L6       67,195,863  0003710596   0004241248
    + 000044.sst  L6       67,196,029  0004241249   0004771076
    + 000045.sst  L6       67,207,959  0004771077   0005301670
    + 000046.sst  L6       67,203,271  0005301671   0005831780
    + 000047.sst  L6       11,225,019  0005831781   0005920000

======== UNIVERSAL totals ========
orders loaded         : 1,500,000
flushes               : 20 (763,020,785 bytes written to L0)
compactions           : 3 (0 trivial moves, 3 merges)
compaction read bytes : 1,226,350,287
compaction write bytes: 1,223,950,521
write amplification   : 2.60  ((flush + compaction writes) / flush writes)
```

6.5 Universal: final SSTables, table properties and index excerpt (`output/universal/final_sstables.txt`)

```
======== UNIVERSAL final SSTables (/Users/avaish/IE-494/lab9/db/universal/CORDERS) ========
orders loaded 1,500,000, load 6.456 s, flush + settle 5.974 s, db directory 760,633,659 bytes
  SSTables final: 13 files, 760,621,019 bytes, L0=1 L6=12
      file-name   level     file-size  smallest-key largest-key
      000035.sst  L0       10,187,938  0005920001   0006000000
      000036.sst  L6       67,204,013  0000000001   0000529860
```

```
      000037.sst  L6       67,201,977  0000529861   0001060352
      000038.sst  L6       67,192,231  0001060353   0001590949
      000039.sst  L6       67,194,260  0001590950   0002120322
      000040.sst  L6       67,206,723  0002120323   0002651335
      000041.sst  L6       67,200,041  0002651360   0003181378
      000042.sst  L6       67,205,695  0003181379   0003710595
      000043.sst  L6       67,195,863  0003710596   0004241248
      000044.sst  L6       67,196,029  0004241249   0004771076
      000045.sst  L6       67,207,959  0004771077   0005301670
      000046.sst  L6       67,203,271  0005301671   0005831780
      000047.sst  L6       11,225,019  0005831781   0005920000

Table properties and index block of each SSTable (first/last 3 index entries; all entries are in index_dump.txt):

  000035.sst  L0  10,187,938 bytes  keys 0005920001 .. 0006000000
    entries 20,000, data blocks 540, data 10,174,586 B, index 12,205 B, raw key 360,000 B, raw value 34,465,440 B, LZ4,
format_version 5
    index block @ 10,174,586 (12,200 B), 540 entries, 540 restart points, user keys, delta-encoded handles; check OK (1
entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0005920163                 0     19,153
            2  0005920291            19,158     18,956
            3  0005920448            38,119     19,057
          ...  (534 more entries)
          538  0005999781         10,131,488     18,521
          539  0005999942         10,150,014     19,282
          540  0006000000         10,169,301      5,280

  000036.sst  L6  67,204,013 bytes  keys 0000000001 .. 0000529860
    entries 132,468, data blocks 3,556, data 67,122,051 B, index 80,821 B, raw key 2,384,424 B, raw value 227,406,124
B, LZ4, format_version 5
    index block @ 67,122,051 (80,816 B), 3,556 entries, 3556 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0000000134                 0     18,935
            2  000000029              18,940     19,043
            3  000000045              37,988     18,995
          ...  (3,550 more entries)
         3554  0000529730         67,082,954     19,080
         3555  000052986          67,102,039     19,083
         3556  0000529860         67,121,127        919

  000037.sst  L6  67,201,977 bytes  keys 0000529861 .. 0001060352
    entries 132,620, data blocks 3,557, data 67,120,070 B, index 80,766 B, raw key 2,387,160 B, raw value 227,501,272
B, LZ4, format_version 5
    index block @ 67,120,070 (80,761 B), 3,557 entries, 3557 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0000530018                 0     19,028
            2  0000530177             19,033     18,746
            3  000053034              37,784     18,900
          ...  (3,551 more entries)
         3555  0001060197         67,081,475     18,676
         3556  000106033          67,100,156     18,951
         3557  0001060352         67,119,112        953

  000038.sst  L6  67,192,231 bytes  keys 0001060353 .. 0001590949
    entries 132,653, data blocks 3,567, data 67,109,993 B, index 81,097 B, raw key 2,387,754 B, raw value 228,088,982
B, LZ4, format_version 5
    index block @ 67,109,993 (81,092 B), 3,567 entries, 3567 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0001060485                 0     18,617
            2  0001060642             18,622     19,005
            3  0001060774             37,632     18,822
          ...  (3,561 more entries)
         3565  0001590787         67,071,324     18,913
         3566  0001590948         67,090,242     19,062
         3567  0001590949         67,109,309        679

  000039.sst  L6  67,194,260 bytes  keys 0001590950 .. 0002120322
    entries 132,341, data blocks 3,567, data 67,112,065 B, index 81,054 B, raw key 2,382,138 B, raw value 228,095,441
B, LZ4, format_version 5
    index block @ 67,112,065 (81,049 B), 3,567 entries, 3567 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0001591136                 0     19,286
            2  0001591266             19,291     18,554
            3  0001591398             37,850     18,911
          ...  (3,561 more entries)
         3565  0002120164         67,073,584     18,986
         3566  0002120321         67,092,575     18,586
         3567  0002120322         67,111,166        894

  000040.sst  L6  67,206,723 bytes  keys 0002120323 .. 0002651335
```

```
    entries 132,757, data blocks 3,567, data 67,124,501 B, index 81,081 B, raw key 2,389,626 B, raw value 228,097,053
B, LZ4, format_version 5
    index block @ 67,124,501 (81,076 B), 3,567 entries, 3567 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0002120482                 0     18,904
            2  0002120642            18,909     18,569
            3  0002120803            37,483     18,937
          ...  (3,561 more entries)
         3565  0002651201         67,086,269     18,520
         3566  0002651334         67,104,794     18,431
         3567  0002651335         67,123,230      1,266

  000041.sst  L6  67,200,041 bytes  keys 0002651360 .. 0003181378
    entries 132,507, data blocks 3,568, data 67,117,813 B, index 81,087 B, raw key 2,385,126 B, raw value 228,141,412
B, LZ4, format_version 5
    index block @ 67,117,813 (81,082 B), 3,568 entries, 3568 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0002651490                 0     19,029
            2  000265163            19,034     19,192
            3  00026518             38,231     18,829
          ...  (3,562 more entries)
         3566  0003181221         67,079,337     18,859
         3567  0003181377         67,098,201     18,596
         3568  0003181378         67,116,802      1,006

  000042.sst  L6  67,205,695 bytes  keys 0003181379 .. 0003710595
    entries 132,305, data blocks 3,568, data 67,123,437 B, index 81,117 B, raw key 2,381,490 B, raw value 228,194,065
B, LZ4, format_version 5
    index block @ 67,123,437 (81,112 B), 3,568 entries, 3568 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0003181537                 0     18,886
            2  0003181665            18,891     18,737
            3  0003181797            37,633     18,652
          ...  (3,562 more entries)
         3566  0003710438         67,085,235     18,604
         3567  0003710594         67,103,844     18,543
         3568  0003710595         67,122,392      1,040

  000043.sst  L6  67,195,863 bytes  keys 0003710596 .. 0004241248
    entries 132,661, data blocks 3,567, data 67,113,671 B, index 81,051 B, raw key 2,387,898 B, raw value 228,137,246
B, LZ4, format_version 5
    index block @ 67,113,671 (81,046 B), 3,567 entries, 3567 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0003710753                 0     18,379
            2  0003710915            18,384     19,053
            3  0003711074            37,442     18,770
          ...  (3,561 more entries)
         3565  0004241092         67,075,150     18,412
         3566  000424123          67,093,567     18,997
         3567  0004241248         67,112,569      1,097

  000044.sst  L6  67,196,029 bytes  keys 0004241249 .. 0004771076
    entries 132,460, data blocks 3,567, data 67,113,808 B, index 81,080 B, raw key 2,384,280 B, raw value 228,127,911
B, LZ4, format_version 5
    index block @ 67,113,808 (81,075 B), 3,567 entries, 3567 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0004241381                 0     18,362
            2  000424152            18,367     18,874
            3  000424168            37,246     19,266
          ...  (3,561 more entries)
         3565  0004770916         67,075,025     18,778
         3566  0004771075         67,093,808     18,713
         3567  0004771076         67,112,526      1,277

  000045.sst  L6  67,207,959 bytes  keys 0004771077 .. 0005301670
    entries 132,650, data blocks 3,566, data 67,125,831 B, index 80,987 B, raw key 2,387,700 B, raw value 228,108,235
B, LZ4, format_version 5
    index block @ 67,125,831 (80,982 B), 3,566 entries, 3566 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0004771235                 0     18,613
            2  0004771366            18,618     18,962
            3  0004771526            37,585     18,997
          ...  (3,560 more entries)
         3564  0005301537         67,087,401     18,749
         3565  000530167          67,106,155     18,653
         3566  0005301670         67,124,813      1,013

  000046.sst  L6  67,203,271 bytes  keys 0005301671 .. 0005831780
    entries 132,526, data blocks 3,567, data 67,121,063 B, index 81,067 B, raw key 2,385,468 B, raw value 228,130,567
```

```
B, LZ4, format_version 5
    index block @ 67,121,063 (81,062 B), 3,567 entries, 3567 restart points, user keys, delta-encoded handles; check OK
(1 entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0005530184                 0     19,161
            2  0005301986            19,166     18,659
            3  0005302116            37,830     18,729
          ...  (3,561 more entries)
         3565  0005831622         67,082,561     18,833
         3566  000583178          67,101,399     18,832
         3567  0005831780         67,120,236        822

  000047.sst  L6  11,225,019 bytes  keys 0005831781 .. 0005920000
    entries 22,052, data blocks 596, data 11,210,426 B, index 13,454 B, raw key 396,936 B, raw value 38,109,655 B, LZ4,
format_version 5
    index block @ 11,210,426 (13,449 B), 596 entries, 596 restart points, user keys, delta-encoded handles; check OK (1
entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0005831937                 0     18,705
            2  0005832096            18,710     18,977
            3  0005832228            37,692     19,048
          ...  (590 more entries)
          594  00059197           11,155,614     18,680
          595  0005919846         11,174,299     18,819
          596  0005920000         11,193,123     17,298
```

6.6 Full index dumps (`index_dump.txt`)

The complete index of every final SSTable is written to `output/leveled/index_dump.txt` (40,451 lines, 2.3 MB) and `output/universal/index_dump.txt` (40,420 lines, 2.3 MB). They are too long to print in full, so here is the start of each file. Every SSTable section has the same layout and lists every index entry.

leveled, first 40 lines:

```
======== LEVELED final SSTables (/Users/avaish/IE-494/lab9/db/leveled/CORDERS) ========
Every index entry: separator key -> data block handle (offset, size).

==== 000026.sst  L0  39,754,692 bytes  keys 0005608001 .. 0005920000
entries 78,000, data blocks 2,103, data 39,705,773 B, index 47,771 B, raw key 1,404,000 B, raw value 134,535,488 B, LZ
4, format_version 5
index block @ 39,705,773 (47,766 B), 2,103 entries, 2103 restart points, user keys, delta-encoded handles; check OK (1
entry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0005608131                 0     18,977
            2  000560829               18,982     19,098
            3  0005608420              38,085     18,310
            4  0005608576              56,400     18,972
            5  0005608740              75,377     18,793
            6  00056089                94,175     19,266
            7  0005609062             113,446     19,077
            8  0005609222             132,528     18,709
            9  0005609378             151,242     19,135
           10  0005609538             170,382     18,498
           11  000560968              188,885     18,853
           12  0005609857             207,743     18,769
           13  000560999              226,517     18,446
           14  0005610144             244,968     18,826
           15  0005610277             263,799     18,499
           16  000561041              282,303     18,383
           17  0005610563             300,691     18,689
           18  0005610722             319,385     19,185
           19  0005610884             338,575     19,439
           20  0005611041             358,019     19,006
           21  0005611172             377,030     18,917
           22  000561131              395,952     18,918
           23  0005611460             414,875     18,856
           24  0005611616             433,736     18,681
           25  000561176              452,422     19,184
           26  0005611905             471,611     18,356
           27  0005612035             489,972     18,886
           28  0005612194             508,863     18,762
           29  0005612352             527,630     19,104
           30  0005612512             546,739     18,876
           31  0005612644             565,620     19,051
           32  0005612801             584,676     18,970
           33  0005612933             603,651     18,939
```

universal, first 40 lines:

```
======== UNIVERSAL final SSTables (/Users/avaish/IE-494/lab9/db/universal/CORDERS) ========
Every index entry: separator key -> data block handle (offset, size).

==== 000035.sst  L0  10,187,938 bytes  keys 0005920001 .. 0006000000
entries 20,000, data blocks 540, data 10,174,586 B, index 12,205 B, raw key 360,000 B, raw value 34,465,440 B, LZ4, for
mat_version 5
index block @ 10,174,586 (12,200 B), 540 entries, 540 restart points, user keys, delta-encoded handles; check OK (1 ent
ry per data block, contiguous, ascending)
        entry  separator-key    block-offset block-size
            1  0005920163                 0     19,153
            2  0005920291            19,158     18,956
            3  0005920448            38,119     19,057
            4  000592058             57,181     18,973
            5  0005920736            76,159     18,977
            6  0005920866            95,141     18,920
            7  0005920997           114,066     19,058
            8  0005921153           133,129     18,819
            9  0005921312           151,953     18,709
           10  0005921446           170,667     18,753
           11  0005921604           189,425     19,144
           12  0005921734           208,574     18,773
           13  0005921890           227,352     19,282
           14  0005922020           246,639     18,943
           15  0005922178           265,587     19,077
           16  0005922336           284,669     19,024
           17  0005922467           303,698     18,789
           18  0005922597           322,492     18,553
           19  0005922754           341,050     18,667
           20  0005922883           359,722     18,261
           21  000592302            377,988     18,821
           22  0005923172           396,814     18,734
           23  0005923331           415,553     18,650
           24  0005923488           434,208     18,888
           25  0005923622           453,101     18,832
           26  0005923781           471,938     18,686
           27  0005923936           490,629     18,836
           28  0005924067           509,470     18,903
           29  0005924226           528,378     19,093
           30  0005924385           547,476     18,839
           31  0005924544           566,320     18,819
           32  0005924678           585,144     19,147
           33  000592481            604,296     18,267
```

7. Source code

All of `lab9/`: line-numbered and syntax-highlighted Java, XML and bash.

Lab09.java

`lab9/src/main/java/pmj/bdp/rocks/lab9/Lab09.java`

```java
package pmj.bdp.rocks.lab9;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

import org.rocksdb.RocksDB;
import org.rocksdb.RocksDBException;

/**
 * Lab 09: compaction in RocksDB (RocksJava).
 *
 * <pre>
 *   java ... Lab09 all  lab8CordersDb dataDir dbDir outputDir
 *   java ... Lab09 dump lab8CordersDb jsonFile
 *   java ... Lab09 load leveled|universal jsonFile dbDir outputDir
 * </pre>
 */
public class Lab09 {
```

```
24
25      public static void main(String[] args) throws Exception {
26          if (args.length < 1) {
27              usage();
28              return;
29          }
30          RocksDB.loadLibrary();
31          switch (args[0]) {
32              case "all" -> all(Path.of(args[1]), Path.of(args[2]), Path.of(args[3]), Path.of(args[4]));
33              case "dump" -> CordersJson.dump(Path.of(args[1]), Path.of(args[2])).print();
34              case "load" -> CompactionStudy.run(CompactionStudy.Style.parse(args[1]), Path.of(args[2]),
35                      Path.of(args[3]), Path.of(args[4]));
36              default -> usage();
37          }
38      }
39
40      private static void all(Path cordersDb, Path dataDir, Path dbDir, Path outDir)
41              throws IOException, RocksDBException, InterruptedException {
42          Path json = dataDir.resolve("corders.json");
43          CordersJson.dump(cordersDb, json).print();
44
45          List<CompactionStudy.Result> results = new ArrayList<>();
46          for (CompactionStudy.Style style : CompactionStudy.Style.values()) {
47              results.add(CompactionStudy.run(style, json, dbDir, outDir));
48          }
49          writeSummary(outDir.resolve("summary.tsv"), results);
50      }
51
52      private static void writeSummary(Path path, List<CompactionStudy.Result> results) throws
IOException {
53          StringBuilder tsv = new
StringBuilder("style\torders\tflushes\tcompactions\ttrivial_moves\tmerges"
54                  + "\tflushed_bytes\tcompaction_read_bytes\tcompaction_written_bytes\twrite_amp"
55                  + "\tfinal_files\tfinal_levels\tfinal_sst_bytes\tload_s\tsettle_s\n");
56          System.out.println();
57          System.out.println("======== Leveled vs Universal ========");
58          System.out.printf(Locale.ROOT, "%-10s %8s %11s %7s %7s %10s %10s  %-14s %15s%n", "style",
"flushes",
59                  "compactions", "trivial", "merges", "write-amp", "final-sst", "levels", "final-bytes");
60          for (CompactionStudy.Result r : results) {
61              SstEventListener.Totals t = r.totals();
62              String levels = SstTables.levels(r.finalFiles());
63              tsv.append(String.format(Locale.ROOT,
"%s\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%.2f\t%d\t%s\t%d\t%.3f\t%.3f%n",
64                      r.style().dir, r.orders(), t.flushes(), t.compactions(), t.trivialMoves(),
65                      t.compactions() - t.trivialMoves(), t.flushedBytes(), t.compactionReadBytes(),
66                      t.compactionWrittenBytes(), t.writeAmplification(), r.finalFiles().size(), levels,
67                      r.sstBytes(), r.loadNanos() / 1e9, r.settleNanos() / 1e9));
68              System.out.printf(Locale.ROOT, "%-10s %8d %11d %7d %7d %10.2f %10d  %-14s %,15d%n",
69                      r.style().dir, t.flushes(), t.compactions(), t.trivialMoves(),
70                      t.compactions() - t.trivialMoves(), t.writeAmplification(), r.finalFiles().size(),
levels,
71                      r.sstBytes());
72          }
73          Files.writeString(path, tsv.toString(), StandardCharsets.UTF_8);
74          System.out.println("summary: " + path);
75      }
76
77      private static void usage() {
78          System.err.println("Usage:");
79          System.err.println("  Lab09 all  <lab8CordersDb> <dataDir> <dbDir> <outputDir>");
80          System.err.println("  Lab09 dump <lab8CordersDb> <jsonFile>");
81          System.err.println("  Lab09 load <leveled|universal> <jsonFile> <dbDir> <outputDir>");
82      }
83  }
```

CordersJson.java

lab9/src/main/java/pmj/bdp/rocks/lab9/CordersJson.java

```java
package pmj.bdp.rocks.lab9;

import java.io.BufferedOutputStream;
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Locale;

import org.rocksdb.Options;
import org.rocksdb.RocksDB;
import org.rocksdb.RocksDBException;
import org.rocksdb.RocksIterator;

/**
 * Step 0: dump Lab 08's CORDERS database to corders.json.
 *
 * The file is one JSON array with one order document per line, so it is valid
 * JSON and can still be streamed line by line when it is loaded back.
 */
final class CordersJson {

    static final int KEY_WIDTH = 10;
    private static final String KEY_FIELD = "{\"O_ORDERKEY\":";
    private static final byte[] NEWLINE = {'\n'};
    private static final byte[] COMMA_NEWLINE = {',', '\n'};

    private CordersJson() {
    }

    static DumpReport dump(Path cordersDb, Path jsonFile) throws RocksDBException, IOException {
        Files.createDirectories(jsonFile.toAbsolutePath().getParent());
        long started = System.nanoTime();
        long orders = 0;
        long bytes = 0;
        String firstKey = null;
        String lastKey = null;
        try (Options options = new Options().setCreateIfMissing(false);
             RocksDB db = RocksDB.openReadOnly(options, cordersDb.toString());
             RocksIterator it = db.newIterator();
             OutputStream out = new BufferedOutputStream(Files.newOutputStream(jsonFile), 1 << 20)) {
            out.write('[');
            for (it.seekToFirst(); it.isValid(); it.next()) {
                out.write(orders == 0 ? NEWLINE : COMMA_NEWLINE);
                byte[] value = it.value();
                out.write(value);
                bytes += value.length;
                lastKey = new String(it.key(), StandardCharsets.US_ASCII);
                if (firstKey == null) {
                    firstKey = lastKey;
                }
                orders++;
            }
            out.write(NEWLINE);
            out.write(']');
            out.write(NEWLINE);
        }
        return new DumpReport(cordersDb, jsonFile, orders, bytes, Files.size(jsonFile),
                firstKey, lastKey, System.nanoTime() - started);
    }

    static BufferedReader open(Path jsonFile) throws IOException {
        return new BufferedReader(new InputStreamReader(Files.newInputStream(jsonFile),
StandardCharsets.UTF_8),
                1 << 20);
    }

    /** Returns the order document on this line, or null for the array brackets. */
    static String document(String line) {
```

```
        if (line.isEmpty() || line.equals("[") || line.equals("]")) {
            return null;
        }
        return line.endsWith(",") ? line.substring(0, line.length() - 1) : line;
    }

    /** 10-digit zero-padded O_ORDERKEY, the same key Lab 08 used. */
    static byte[] key(String document) {
        if (!document.startsWith(KEY_FIELD)) {
            throw new IllegalArgumentException("document does not start with O_ORDERKEY: "
                    + document.substring(0, Math.min(60, document.length())));
        }
        int start = KEY_FIELD.length();
        int end = start;
        while (end < document.length() && Character.isDigit(document.charAt(end))) {
            end++;
        }
        int digits = end - start;
        if (digits == 0 || digits > KEY_WIDTH) {
            throw new IllegalArgumentException("bad O_ORDERKEY in " + document.substring(0, end));
        }
        byte[] key = new byte[KEY_WIDTH];
        int pad = KEY_WIDTH - digits;
        for (int i = 0; i < KEY_WIDTH; i++) {
            key[i] = i < pad ? (byte) '0' : (byte) document.charAt(start + i - pad);
        }
        return key;
    }

    record DumpReport(Path source, Path file, long orders, long valueBytes, long fileBytes,
                      String firstKey, String lastKey, long nanos) {

        void print() {
            System.out.println();
            System.out.println("======== Dump CORDERS -> corders.json ========");
            System.out.println("source rocksdb : " + source);
            System.out.println("json file      : " + file);
            System.out.printf(Locale.ROOT, "orders written : %,d%n", orders);
            System.out.printf(Locale.ROOT, "document bytes : %,d%n", valueBytes);
            System.out.printf(Locale.ROOT, "file size      : %,d bytes (%.1f MiB)%n",
                    fileBytes, fileBytes / (1024.0 * 1024.0));
            System.out.println("key range      : " + firstKey + " .. " + lastKey);
            System.out.printf(Locale.ROOT, "dump time      : %.3f s%n", nanos / 1_000_000_000.0);
        }
    }
}
```

CompactionStudy.java

lab9/src/main/java/pmj/bdp/rocks/lab9/CompactionStudy.java

```
package pmj.bdp.rocks.lab9;

import java.io.BufferedOutputStream;
import java.io.BufferedReader;
import java.io.IOException;
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Set;

import org.rocksdb.BlockBasedTableConfig;
import org.rocksdb.CompactionOptionsUniversal;
import org.rocksdb.CompactionStyle;
import org.rocksdb.CompressionType;
import org.rocksdb.FlushOptions;
import org.rocksdb.InfoLogLevel;
```

```java
import org.rocksdb.Options;
import org.rocksdb.RocksDB;
import org.rocksdb.RocksDBException;
import org.rocksdb.SstFileReader;
import org.rocksdb.TableProperties;
import org.rocksdb.WriteBatch;
import org.rocksdb.WriteOptions;

/**
 * Steps 1 and 2 for one compaction style: load every order from corders.json
 * into a fresh CORDERS database while {@link SstEventListener} logs each flush
 * and compaction, then list the final SSTables and dump their indexes.
 */
final class CompactionStudy {

    // Same settings for both styles; only the compaction style changes.
    static final long WRITE_BUFFER_SIZE = 128L * 1024 * 1024;
    static final int MAX_WRITE_BUFFERS = 2;
    static final int L0_COMPACTION_TRIGGER = 4;
    static final long TARGET_FILE_SIZE = 64L * 1024 * 1024;
    static final long MAX_BYTES_FOR_LEVEL_BASE = 256L * 1024 * 1024;
    static final long BLOCK_SIZE = 64L * 1024;
    static final int FORMAT_VERSION = 5;
    static final int BACKGROUND_JOBS = 2;
    static final int BATCH_ROWS = 2_000;
    static final int INDEX_EXCERPT = 3;

    enum Style {
        LEVELED("leveled"), UNIVERSAL("universal");

        final String dir;

        Style(String dir) {
            this.dir = dir;
        }

        static Style parse(String name) {
            return switch (name.toLowerCase(Locale.ROOT)) {
                case "leveled", "level", "levelled" -> LEVELED;
                case "universal" -> UNIVERSAL;
                default -> throw new IllegalArgumentException("style must be leveled or universal: " +
name);
            };
        }
    }

    private CompactionStudy() {
    }

    static Result run(Style style, Path jsonFile, Path dbRoot, Path outRoot)
            throws IOException, RocksDBException, InterruptedException {
        Path dbPath = dbRoot.resolve(style.dir).resolve("CORDERS");
        Path outDir = outRoot.resolve(style.dir);
        deleteDirectory(dbPath);
        Files.createDirectories(dbPath);
        Files.createDirectories(outDir);

        System.out.println();
        System.out.println("======== " + style + " compaction: load corders.json -> " + dbPath + "
========");
        List<SstTables.Sst> finalFiles;
        SstEventListener.Totals totals;
        long orders = 0;
        long loadNanos;
        long settleNanos;

        try (PrintStream events = printStream(outDir.resolve("events.txt"))) {
            events.println(style + " compaction: every flush and compaction while loading
corders.json");
            events.println("rocksdb path: " + dbPath);
            events.println(config(style));
            events.println("'*' marks compaction inputs, '+' marks files written (or moved) by the
job.");
```

```
 90
 91              try (CompactionOptionsUniversal universal = new CompactionOptionsUniversal();
 92                   SstEventListener listener = new SstEventListener(events);
 93                   Options options = options(style, universal, listener);
 94                   RocksDB db = RocksDB.open(options, dbPath.toString());
 95                   WriteOptions writeOptions = new WriteOptions().setDisableWAL(true);
 96                   WriteBatch batch = new WriteBatch();
 97                   BufferedReader in = CordersJson.open(jsonFile)) {
 98
 99                  long started = System.nanoTime();
100                  String line;
101                  int pending = 0;
102                  while ((line = in.readLine()) != null) {
103                      String doc = CordersJson.document(line);
104                      if (doc == null) {
105                          continue;
106                      }
107                      batch.put(CordersJson.key(doc), doc.getBytes(StandardCharsets.UTF_8));
108                      orders++;
109                      if (++pending == BATCH_ROWS) {
110                          db.write(writeOptions, batch);
111                          batch.clear();
112                          pending = 0;
113                      }
114                      if (orders % 250_000 == 0) {
115                          System.out.printf(Locale.ROOT, "  orders %,d in %.1fs, live SSTables %s%n",
orders,
116                                  (System.nanoTime() - started) / 1_000_000_000.0,
117                                  SstTables.levels(SstTables.snapshot(db)));
118                      }
119                  }
120                  if (pending > 0) {
121                      db.write(writeOptions, batch);
122                  }
123                  loadNanos = System.nanoTime() - started;
124
125                  // Flush the last memtable, then let background compactions finish on their own.
126                  long settleStart = System.nanoTime();
127                  try (FlushOptions flush = new FlushOptions().setWaitForFlush(true)) {
128                      db.flush(flush);
129                  }
130                  waitForBackgroundWork(db);
131                  settleNanos = System.nanoTime() - settleStart;
132
133                  finalFiles = SstTables.snapshot(db);
134                  totals = listener.totals();
135                  synchronized (listener) {
136                      events.println();
137                      events.println("======== " + style + " totals ========");
138                      events.printf(Locale.ROOT, "orders loaded         : %,d%n", orders);
139                      totals.print(events);
140                  }
141              }
142          }
143
144          totals.print(System.out);
145          Result result = new Result(style, dbPath, orders, loadNanos, settleNanos, totals, finalFiles,
146                  directoryBytes(dbPath));
147          writeFinal(result, outDir);
148          return result;
149      }
150
151      static String config(Style style) {
152          return String.format(Locale.ROOT,
153                  "options: compaction_style=%s%s, write_buffer_size=%d MiB x %d,
level0_file_num_compaction_trigger=%d,"
154                          + " target_file_size_base=%d MiB, max_bytes_for_level_base=%d MiB,
compression=LZ4,"
155                          + " block_size=%d KiB, format_version=%d, index compression off,
max_background_jobs=%d,"
156                          + " WAL off, batch=%,d",
157                  style == Style.LEVELED ? "kCompactionStyleLevel" : "kCompactionStyleUniversal",
158                  style == Style.LEVELED ? " (level_compaction_dynamic_level_bytes=true)"
```

```
                        : " (size_ratio=1, min_merge_width=2, max_size_amplification_percent=200)",
                WRITE_BUFFER_SIZE >> 20, MAX_WRITE_BUFFERS, L0_COMPACTION_TRIGGER, TARGET_FILE_SIZE >>
20,
                MAX_BYTES_FOR_LEVEL_BASE >> 20, BLOCK_SIZE >> 10, FORMAT_VERSION, BACKGROUND_JOBS,
BATCH_ROWS);
    }

    private static Options options(Style style, CompactionOptionsUniversal universal, SstEventListener
listener) {
        BlockBasedTableConfig table = new BlockBasedTableConfig()
                .setBlockSize(BLOCK_SIZE)
                .setFormatVersion(FORMAT_VERSION)
                .setEnableIndexCompression(false);
        Options options = new Options()
                .setCreateIfMissing(true)
                .setCompressionType(CompressionType.LZ4_COMPRESSION)
                .setWriteBufferSize(WRITE_BUFFER_SIZE)
                .setMaxWriteBufferNumber(MAX_WRITE_BUFFERS)
                .setLevel0FileNumCompactionTrigger(L0_COMPACTION_TRIGGER)
                .setTargetFileSizeBase(TARGET_FILE_SIZE)
                .setMaxBytesForLevelBase(MAX_BYTES_FOR_LEVEL_BASE)
                .setMaxBackgroundJobs(BACKGROUND_JOBS)
                .setTableFormatConfig(table)
                .setListeners(List.of(listener))
                .setInfoLogLevel(InfoLogLevel.ERROR_LEVEL);
        if (style == Style.LEVELED) {
            options.setCompactionStyle(CompactionStyle.LEVEL);
            options.setLevelCompactionDynamicLevelBytes(true);
        } else {
            universal.setSizeRatio(1);
            universal.setMinMergeWidth(2);
            universal.setMaxSizeAmplificationPercent(200);
            options.setCompactionStyle(CompactionStyle.UNIVERSAL);
            options.setCompactionOptionsUniversal(universal);
        }
        return options;
    }

    /** RocksJava has no WaitForCompact, so poll until nothing is running or pending. */
    private static void waitForBackgroundWork(RocksDB db) throws RocksDBException,
InterruptedException {
        int quiet = 0;
        while (quiet < 5) {
            Thread.sleep(200);
            boolean busy = db.getLongProperty("rocksdb.num-running-compactions") > 0
                    || db.getLongProperty("rocksdb.num-running-flushes") > 0
                    || db.getLongProperty("rocksdb.compaction-pending") > 0
                    || db.getLongProperty("rocksdb.mem-table-flush-pending") > 0;
            quiet = busy ? 0 : quiet + 1;
        }
    }

    /** Step 2: final SSTables, their table properties, and their index blocks. */
    private static void writeFinal(Result r, Path outDir) throws IOException, RocksDBException {
        try (PrintStream fin = printStream(outDir.resolve("final_sstables.txt"));
             PrintStream dump = printStream(outDir.resolve("index_dump.txt"));
             Options options = new Options()) {
            String title = "======== " + r.style() + " final SSTables (" + r.dbPath() + ") ========";
            fin.println(title);
            fin.printf(Locale.ROOT, "orders loaded %,d, load %.3f s, flush + settle %.3f s, db
directory %,d bytes%n",
                    r.orders(), r.loadNanos() / 1e9, r.settleNanos() / 1e9, r.directoryBytes());
            SstTables.print(fin, "SSTables final", r.finalFiles(), Set.of(), ' ');
            fin.println();
            fin.println("Table properties and index block of each SSTable (first/last " +
INDEX_EXCERPT
                    + " index entries; all entries are in index_dump.txt):");
            dump.println(title);
            dump.println("Every index entry: separator key -> data block handle (offset, size).");

            for (SstTables.Sst f : r.finalFiles()) {
                Path file = r.dbPath().resolve(f.name());
                TableProperties p;
```

```
226                 try (SstFileReader reader = new SstFileReader(options)) {
227                     reader.open(file.toString());
228                     p = reader.getTableProperties();
229                 }
230                 SstIndex.Index index = SstIndex.read(file, p.getIndexKeyIsUserKey() != 0,
231                         p.getIndexValueIsDeltaEncoded() != 0);
232                 String check = check(index, p);
233                 String head = String.format(Locale.ROOT,
234                         "%s  L%d  %,d bytes  keys %s .. %s", f.name(), f.level(), f.size(),
f.smallestKey(),
235                         f.largestKey());
236                 String props = String.format(Locale.ROOT,
237                         "entries %,d, data blocks %,d, data %,d B, index %,d B, raw key %,d B, raw
value %,d B,"
238                                 + " %s, format_version %d",
239                         p.getNumEntries(), p.getNumDataBlocks(), p.getDataSize(), p.getIndexSize(),
240                         p.getRawKeySize(), p.getRawValueSize(), p.getCompressionName(),
index.formatVersion());
241                 String layout = String.format(Locale.ROOT,
242                         "index block @ %,d (%,d B), %,d entries, %d restart points, %s keys, %s; %s",
243                         index.index().offset(), index.index().size(), index.entries().size(),
index.restarts(),
244                         p.getIndexKeyIsUserKey() != 0 ? "user" : "internal",
245                         p.getIndexValueIsDeltaEncoded() != 0 ? "delta-encoded handles" : "full
handles", check);
246
247                 fin.println();
248                 fin.println("  " + head);
249                 fin.println("    " + props);
250                 fin.println("    " + layout);
251                 fin.printf(Locale.ROOT, "      %7s  %-14s %14s %10s%n", "entry", "separator-key",
"block-offset",
252                         "block-size");
253                 List<SstIndex.Entry> entries = index.entries();
254                 for (int i = 0; i < entries.size(); i++) {
255                     if (i == INDEX_EXCERPT && entries.size() > 2 * INDEX_EXCERPT) {
256                         fin.printf(Locale.ROOT, "      %7s  (%,d more entries)%n", "...",
257                                 entries.size() - 2 * INDEX_EXCERPT);
258                         i = entries.size() - INDEX_EXCERPT;
259                     }
260                     printEntry(fin, i, entries.get(i));
261                 }
262
263                 dump.println();
264                 dump.println("==== " + head);
265                 dump.println(props);
266                 dump.println(layout);
267                 dump.printf(Locale.ROOT, "      %7s  %-14s %14s %10s%n", "entry", "separator-key",
"block-offset",
268                         "block-size");
269                 for (int i = 0; i < entries.size(); i++) {
270                     printEntry(dump, i, entries.get(i));
271                 }
272             }
273         }
274         System.out.println();
275         System.out.print(Files.readString(outDir.resolve("final_sstables.txt")));
276     }
277
278     private static void printEntry(PrintStream out, int i, SstIndex.Entry e) {
279         String key = e.sequence() < 0 ? e.keyText() : e.keyText() + "@" + e.sequence();
280         out.printf(Locale.ROOT, "      %7d  %-14s %,14d %,10d%n", i + 1, key, e.block().offset(),
e.block().size());
281     }
282
283     /** Index sanity: one entry per data block, blocks contiguous from offset 0, keys ascending. */
284     private static String check(SstIndex.Index index, TableProperties p) {
285         List<SstIndex.Entry> entries = index.entries();
286         if (entries.size() != p.getNumDataBlocks()) {
287             return "CHECK FAILED: " + entries.size() + " entries but " + p.getNumDataBlocks() + " data
blocks";
288         }
289         long expected = 0;
```

```
        String previous = null;
        for (SstIndex.Entry e : entries) {
            if (e.block().offset() != expected) {
                return "CHECK FAILED: block at " + e.block().offset() + ", expected " + expected;
            }
            expected = e.block().offset() + e.block().size() + SstIndex.BLOCK_TRAILER;
            if (previous != null && previous.compareTo(e.keyText()) >= 0) {
                return "CHECK FAILED: separator " + e.keyText() + " not after " + previous;
            }
            previous = e.keyText();
        }
        return "check OK (1 entry per data block, contiguous, ascending)";
    }

    private static PrintStream printStream(Path path) throws IOException {
        return new PrintStream(new BufferedOutputStream(Files.newOutputStream(path), 1 << 16), false,
                StandardCharsets.UTF_8);
    }

    static long directoryBytes(Path root) throws IOException {
        try (var walk = Files.walk(root)) {
            long total = 0;
            for (Path p : walk.filter(Files::isRegularFile).toList()) {
                total += Files.size(p);
            }
            return total;
        }
    }

    static void deleteDirectory(Path root) throws IOException {
        if (!Files.exists(root)) {
            return;
        }
        try (var walk = Files.walk(root)) {
            for (Path p : walk.sorted(Comparator.reverseOrder()).toList()) {
                Files.deleteIfExists(p);
            }
        }
    }

    record Result(Style style, Path dbPath, long orders, long loadNanos, long settleNanos,
                  SstEventListener.Totals totals, List<SstTables.Sst> finalFiles, long directoryBytes)
{

        long sstBytes() {
            long total = 0;
            for (SstTables.Sst f : finalFiles) {
                total += f.size();
            }
            return total;
        }
    }
}
```

SstEventListener.java

lab9/src/main/java/pmj/bdp/rocks/lab9/SstEventListener.java

```
package pmj.bdp.rocks.lab9;

import java.io.PrintStream;
import java.util.HashSet;
import java.util.List;
import java.util.Locale;
import java.util.Set;

import org.rocksdb.AbstractEventListener;
import org.rocksdb.CompactionJobInfo;
import org.rocksdb.CompactionJobStats;
import org.rocksdb.FlushJobInfo;
import org.rocksdb.RocksDB;
```

```
 14
 15 /**
 16  * Catches every flush and compaction. For each one it prints the live
 17  * SSTables before (on *Begin) and after (on *Completed) the job.
 18  *
 19  * Callbacks run on RocksDB background threads, so every event is written as
 20  * one synchronized block and exceptions never escape back into JNI.
 21  */
 22 final class SstEventListener extends AbstractEventListener {
 23
 24     private final PrintStream out;
 25     private final long started = System.nanoTime();
 26     private int events;
 27     private int flushes;
 28     private int compactions;
 29     private int trivialMoves;
 30     private long flushedBytes;
 31     private long compactionReadBytes;
 32     private long compactionWrittenBytes;
 33
 34     SstEventListener(PrintStream out) {
 35         super(EnabledEventCallback.ON_FLUSH_BEGIN,
 36                 EnabledEventCallback.ON_FLUSH_COMPLETED,
 37                 EnabledEventCallback.ON_COMPACTION_BEGIN,
 38                 EnabledEventCallback.ON_COMPACTION_COMPLETED);
 39         this.out = out;
 40     }
 41
 42     @Override
 43     public void onFlushBegin(RocksDB db, FlushJobInfo info) {
 44         guard("onFlushBegin", () -> {
 45             List<SstTables.Sst> before = SstTables.snapshot(db);
 46             // The output file and seqno range are only known once the flush completes.
 47             header(String.format(Locale.ROOT, "FLUSH BEGIN       job=%d reason=%s",
 48                     info.getJobId(), info.getFlushReason()));
 49             SstTables.print(out, "SSTables before", before, Set.of(), ' ');
 50         });
 51     }
 52
 53     @Override
 54     public void onFlushCompleted(RocksDB db, FlushJobInfo info) {
 55         guard("onFlushCompleted", () -> {
 56             List<SstTables.Sst> after = SstTables.snapshot(db);
 57             String file = SstTables.fileName(info.getFilePath());
 58             long size = SstTables.sizeOf(after, file);
 59             flushes++;
 60             flushedBytes += size;
 61             header(String.format(Locale.ROOT, "FLUSH COMPLETED   job=%d flush #%d wrote %s (%,d bytes)
to L0, seqno %,d..%,d%s",
 62                     info.getJobId(), flushes, file, size, info.getSmallestSeqno(),
info.getLargestSeqno(),
 63                     info.isTriggeredWritesStop() ? " [writes stopped]"
 64                             : info.isTriggeredWritesSlowdown() ? " [writes slowed]" : ""));
 65             SstTables.print(out, "SSTables after", after, Set.of(file), '+');
 66         });
 67     }
 68
 69     @Override
 70     public void onCompactionBegin(RocksDB db, CompactionJobInfo info) {
 71         guard("onCompactionBegin", () -> {
 72             List<SstTables.Sst> before = SstTables.snapshot(db);
 73             Set<String> inputs = names(info.inputFiles());
 74             header(String.format(Locale.ROOT, "COMPACTION BEGIN  job=%d reason=%s L%d -> L%d, %d input
files",
 75                     info.jobId(), info.compactionReason(), info.baseInputLevel(), info.outputLevel(),
 76                     inputs.size()));
 77             SstTables.print(out, "SSTables before ('*' = input)", before, inputs, '*');
 78         });
 79     }
 80
 81     @Override
 82     public void onCompactionCompleted(RocksDB db, CompactionJobInfo info) {
 83         guard("onCompactionCompleted", () -> {
```

```
 84              List<SstTables.Sst> after = SstTables.snapshot(db);
 85              Set<String> inputs = names(info.inputFiles());
 86              Set<String> outputs = names(info.outputFiles());
 87              boolean trivial = inputs.equals(outputs);
 88              long read;
 89              long written;
 90              try (CompactionJobStats stats = info.stats()) {
 91                  read = stats == null ? 0 : stats.totalInputBytes();
 92                  written = stats == null ? 0 : stats.totalOutputBytes();
 93              }
 94              compactions++;
 95              if (trivial) {
 96                  trivialMoves++;
 97              }
 98              compactionReadBytes += read;
 99              compactionWrittenBytes += written;
100              header(String.format(Locale.ROOT,
101                      "COMPACTION DONE   job=%d compaction #%d L%d -> L%d %s: %d in, %d out, read %,d,
wrote %,d bytes",
102                      info.jobId(), compactions, info.baseInputLevel(), info.outputLevel(),
103                      trivial ? "TRIVIAL MOVE (no rewrite)" : "merge",
104                      inputs.size(), outputs.size(), read, written));
105              SstTables.print(out, "SSTables after ('+' = output)", after, outputs, '+');
106          });
107      }
108
109      synchronized Totals totals() {
110          return new Totals(flushes, compactions, trivialMoves, flushedBytes, compactionReadBytes,
111                  compactionWrittenBytes);
112      }
113
114      record Totals(int flushes, int compactions, int trivialMoves, long flushedBytes,
115                    long compactionReadBytes, long compactionWrittenBytes) {
116
117          /** SST bytes written by flushes and compactions, per byte flushed. */
118          double writeAmplification() {
119              return flushedBytes == 0 ? 0 : (flushedBytes + compactionWrittenBytes) / (double)
flushedBytes;
120          }
121
122          void print(PrintStream out) {
123              out.printf(Locale.ROOT, "flushes              : %d (%,d bytes written to L0)%n", flushes,
flushedBytes);
124              out.printf(Locale.ROOT, "compactions          : %d (%d trivial moves, %d merges)%n",
125                      compactions, trivialMoves, compactions - trivialMoves);
126              out.printf(Locale.ROOT, "compaction read bytes : %,d%n", compactionReadBytes);
127              out.printf(Locale.ROOT, "compaction write bytes: %,d%n", compactionWrittenBytes);
128              out.printf(Locale.ROOT, "write amplification   : %.2f  ((flush + compaction writes) /
flush writes)%n",
129                      writeAmplification());
130          }
131      }
132
133      private void header(String text) {
134          events++;
135          double t = (System.nanoTime() - started) / 1_000_000_000.0;
136          out.println();
137          out.printf(Locale.ROOT, "[event %03d  t=%7.3fs] %s%n", events, t, text);
138      }
139
140      private synchronized void guard(String callback, Runnable body) {
141          try {
142              body.run();
143              out.flush();
144          } catch (Throwable e) {
145              out.println("!! " + callback + " failed: " + e);
146          }
147      }
148
149      private static Set<String> names(List<String> paths) {
150          Set<String> names = new HashSet<>();
151          for (String p : paths) {
152              names.add(SstTables.fileName(p));
```

```
        }
        return names;
    }
}
```

SstTables.java

lab9/src/main/java/pmj/bdp/rocks/lab9/SstTables.java

```java
package pmj.bdp.rocks.lab9;

import java.io.PrintStream;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import java.util.TreeMap;

import org.rocksdb.LiveFileMetaData;
import org.rocksdb.RocksDB;

/** One row per live SSTable: file-name, file-level, file-size, smallest-key, largest-key. */
final class SstTables {

    private SstTables() {
    }

    record Sst(String name, int level, long size, String smallestKey, String largestKey, long entries)
{
    }

    static List<Sst> snapshot(RocksDB db) {
        List<Sst> files = new ArrayList<>();
        for (LiveFileMetaData f : db.getLiveFilesMetaData()) {
            files.add(new Sst(fileName(f.fileName()), f.level(), f.size(),
                    text(f.smallestKey()), text(f.largestKey()), f.numEntries()));
        }
        files.sort(Comparator.comparingInt(Sst::level)
                .thenComparing(Sst::smallestKey)
                .thenComparing(Sst::name));
        return files;
    }

    /**
     * Prints the table. A file whose name is in {@code marked} gets {@code mark}
     * in the first column ('*' = compaction input, '+' = written by this job).
     */
    static void print(PrintStream out, String title, List<Sst> files, Set<String> marked, char mark) {
        long total = 0;
        for (Sst f : files) {
            total += f.size();
        }
        out.printf(Locale.ROOT, "  %s: %d files, %,d bytes, %s%n", title, files.size(), total,
levels(files));
        if (files.isEmpty()) {
            return;
        }
        out.printf(Locale.ROOT, "      %-11s %-5s %13s  %-12s %s%n",
                "file-name", "level", "file-size", "smallest-key", "largest-key");
        for (Sst f : files) {
            char m = marked.contains(f.name()) ? mark : ' ';
            out.printf(Locale.ROOT, "    %c %-11s L%-4d %,13d  %-12s %s%n",
                    m, f.name(), f.level(), f.size(), f.smallestKey(), f.largestKey());
        }
    }

    /** "L0=2 L6=5" style summary of how many files sit at each level. */
    static String levels(List<Sst> files) {
```

```
        Map<Integer, Integer> perLevel = new TreeMap<>();
        for (Sst f : files) {
            perLevel.merge(f.level(), 1, Integer::sum);
        }
        if (perLevel.isEmpty()) {
            return "no levels";
        }
        StringBuilder sb = new StringBuilder();
        perLevel.forEach((level, n) -> sb.append(sb.isEmpty() ? "" : " ").append('L').append(level)
                .append('=').append(n));
        return sb.toString();
    }

    static long sizeOf(List<Sst> files, String name) {
        for (Sst f : files) {
            if (f.name().equals(name)) {
                return f.size();
            }
        }
        return 0;
    }

    /** "/path/to/000123.sst" or "/000123.sst" becomes "000123.sst". */
    static String fileName(String path) {
        int slash = path.lastIndexOf('/');
        return slash < 0 ? path : path.substring(slash + 1);
    }

    private static String text(byte[] key) {
        return key == null ? "(null)" : new String(key, StandardCharsets.US_ASCII);
    }
}
```

SstIndex.java

lab9/src/main/java/pmj/bdp/rocks/lab9/SstIndex.java

```
package pmj.bdp.rocks.lab9;

import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.ByteOrder;
import java.nio.channels.FileChannel;
import java.nio.charset.StandardCharsets;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * Reads the index block of a RocksDB block-based SSTable.
 *
 * RocksJava has no API for the index block, so this decodes the on-disk
 * format directly (RocksDB wiki "Rocksdb BlockBasedTable Format"):
 *
 * <pre>
 *   [data block 1] ... [data block N] [meta blocks] [metaindex block] [index block] [footer]
 *   footer (53 bytes, format_version 1..5):
 *     checksum type (1) | metaindex handle | index handle | padding to 41 bytes
 *     | format_version (fixed32) | magic number (fixed64)
 *   block = contents + trailer (compression type 1 byte + checksum 4 bytes)
 *   index entry = separator key -> BlockHandle(offset, size) of one data block
 * </pre>
 *
 * Tables are written with format_version=5, an uncompressed binary-search
 * index, and the default index_block_restart_interval of 1.
 */
final class SstIndex {

    static final long BLOCK_BASED_MAGIC = 0x88e241b785f4cff7L;
```

```java
    static final int FOOTER_LENGTH = 53;
    static final int BLOCK_TRAILER = 5;

    private SstIndex() {
    }

    record Handle(long offset, long size) {
    }

    /** One index entry. {@code sequence} is -1 when the index stores user keys. */
    record Entry(byte[] key, long sequence, Handle block) {
        String keyText() {
            return new String(key, StandardCharsets.US_ASCII);
        }
    }

    record Index(int formatVersion, Handle metaindex, Handle index, int restarts, List<Entry> entries)
{
    }

    static Index read(Path sst, boolean keyIsUserKey, boolean valueDeltaEncoded) throws IOException {
        try (FileChannel ch = FileChannel.open(sst, StandardOpenOption.READ)) {
            long length = ch.size();
            ByteBuffer footer = read(ch, length - FOOTER_LENGTH, FOOTER_LENGTH);
            long magic = footer.getLong(FOOTER_LENGTH - 8);
            if (magic != BLOCK_BASED_MAGIC) {
                throw new IOException(sst + ": not a block-based table (magic " +
Long.toHexString(magic) + ")");
            }
            int formatVersion = footer.getInt(FOOTER_LENGTH - 12);
            if (formatVersion < 1 || formatVersion > 5) {
                throw new IOException(sst + ": footer format_version " + formatVersion + " is not
supported");
            }
            footer.position(1); // skip checksum type
            Handle metaindex = new Handle(varint64(footer), varint64(footer));
            Handle index = new Handle(varint64(footer), varint64(footer));

            int size = Math.toIntExact(index.size());
            ByteBuffer block = read(ch, index.offset(), size + BLOCK_TRAILER);
            int compression = block.get(size) & 0xff;
            if (compression != 0) {
                throw new IOException(sst + ": index block is compressed (type " + compression + ")");
            }
            block.limit(size);
            int restarts = block.getInt(size - 4) & 0x7fffffff;
            List<Entry> entries = entries(block, size - 4 - 4 * restarts, keyIsUserKey,
valueDeltaEncoded);
            return new Index(formatVersion, metaindex, index, restarts, entries);
        }
    }

    /**
     * Entry layout: shared (varint32) | non_shared (varint32) | [value length (varint32)]
     * | key delta | value. With delta-encoded values (format_version >= 4) there is no
     * value length; an entry that shares no key bytes stores a full BlockHandle, and
     * any other entry stores only the signed size change from the previous block.
     */
    private static List<Entry> entries(ByteBuffer block, int end, boolean keyIsUserKey, boolean
valueDeltaEncoded) {
        List<Entry> entries = new ArrayList<>();
        byte[] key = new byte[0];
        Handle previous = null;
        block.position(0);
        while (block.position() < end) {
            int shared = (int) varint64(block);
            int nonShared = (int) varint64(block);
            int valueLength = valueDeltaEncoded ? -1 : (int) varint64(block);
            byte[] next = Arrays.copyOf(key, shared + nonShared);
            block.get(next, shared, nonShared);
            key = next;

            Handle handle;
```

```
            if (valueDeltaEncoded && shared > 0 && previous != null) {
                long delta = zigzag(varint64(block));
                handle = new Handle(previous.offset() + previous.size() + BLOCK_TRAILER,
previous.size() + delta);
            } else {
                int valueStart = block.position();
                handle = new Handle(varint64(block), varint64(block));
                if (valueLength >= 0) {
                    block.position(valueStart + valueLength);
                }
            }
            previous = handle;

            if (keyIsUserKey) {
                entries.add(new Entry(key, -1, handle));
            } else {
                // Internal key: user key + fixed64 (sequence << 8 | value type).
                int userLength = key.length - 8;
                long packed = ByteBuffer.wrap(key, userLength,
8).order(ByteOrder.LITTLE_ENDIAN).getLong();
                entries.add(new Entry(Arrays.copyOf(key, userLength), packed >>> 8, handle));
            }
        }
        return entries;
    }

    private static ByteBuffer read(FileChannel ch, long offset, int length) throws IOException {
        ByteBuffer buf = ByteBuffer.allocate(length).order(ByteOrder.LITTLE_ENDIAN);
        while (buf.hasRemaining()) {
            if (ch.read(buf, offset + buf.position()) < 0) {
                throw new IOException("unexpected end of file at " + (offset + buf.position()));
            }
        }
        return buf.flip();
    }

    private static long varint64(ByteBuffer buf) {
        long result = 0;
        for (int shift = 0; shift < 64; shift += 7) {
            byte b = buf.get();
            result |= (long) (b & 0x7f) << shift;
            if ((b & 0x80) == 0) {
                return result;
            }
        }
        throw new IllegalStateException("varint longer than 10 bytes");
    }

    private static long zigzag(long n) {
        return (n >>> 1) ^ -(n & 1);
    }
}
```

pom.xml

lab9/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>pmj.bdp.rocks</groupId>
    <artifactId>lab9</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>

    <properties>
```

```
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>17</maven.compiler.release>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.rocksdb</groupId>
            <artifactId>rocksdbjni</artifactId>
            <version>10.10.1.1</version>
        </dependency>
    </dependencies>

    <build>
        <finalName>lab9</finalName>
        <plugins>
            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>3.5.0</version>
            </plugin>
        </plugins>
    </build>

</project>
```

run_all.sh

lab9/run_all.sh

```
#!/usr/bin/env bash
set -euo pipefail
ROOT="$(cd "$(dirname "$0")" && pwd)"
cd "$ROOT"
LAB8="$(cd "$ROOT/../lab8" && pwd)"
CORDERS="${1:-$LAB8/db/CORDERS_zstd}"
DATA="$ROOT/data"
DB="$ROOT/db"
OUT="$ROOT/output"

# Lab 08 output CORDERS is the input. Rebuild it if it is missing.
if [ ! -f "$CORDERS/CURRENT" ]; then
  (cd "$LAB8" && mvn -q -DskipTests package \
    && mvn -q exec:java -Dexec.mainClass=pmj.bdp.rocks.lab8.Lab08 \
         -Dexec.args="load $LAB8/data $LAB8/db $DATA/lab8_run" \
    && mvn -q exec:java -Dexec.mainClass=pmj.bdp.rocks.lab8.Lab08 \
         -Dexec.args="merge $LAB8/db zstd $DATA/lab8_run")
fi

mvn -q -DskipTests package
rm -rf "$OUT"
mkdir -p "$OUT" "$DATA"
# exec:exec forks a plain JVM. Under exec:java, rocksdbjni sits in Maven's
# classloader and the listener callbacks (native threads) cannot find its classes.
mvn -q exec:exec -Dexec.executable=java \
  -Dexec.args="--enable-native-access=ALL-UNNAMED -cp %classpath pmj.bdp.rocks.lab9.Lab09 all $CORDERS
$DATA $DB $OUT" \
  | tee "$OUT/run_log.txt"
```